



Computer Vision for Autonomous Vehicles

Machine Learning and Object detection + 3D construction

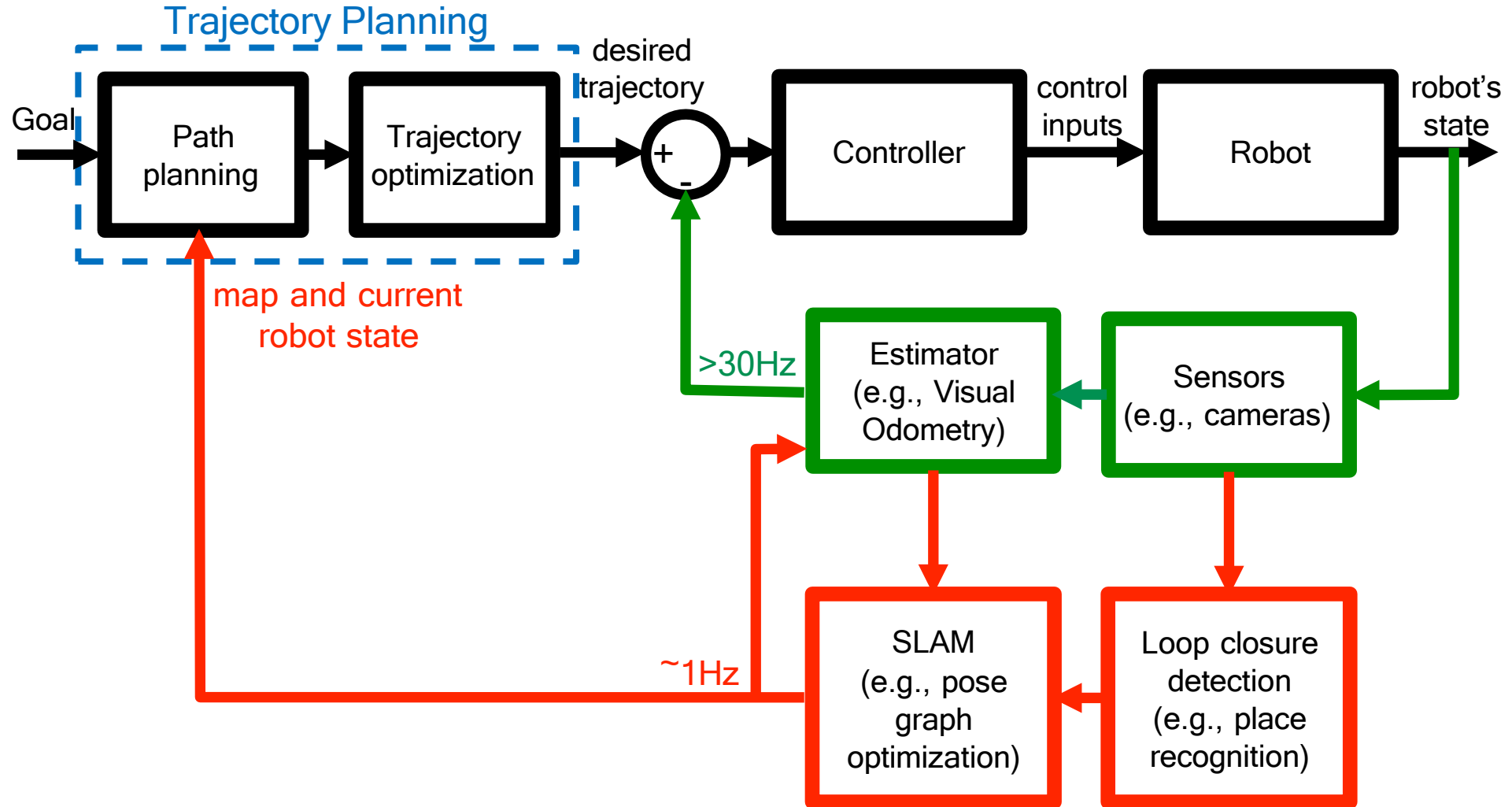
Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2025



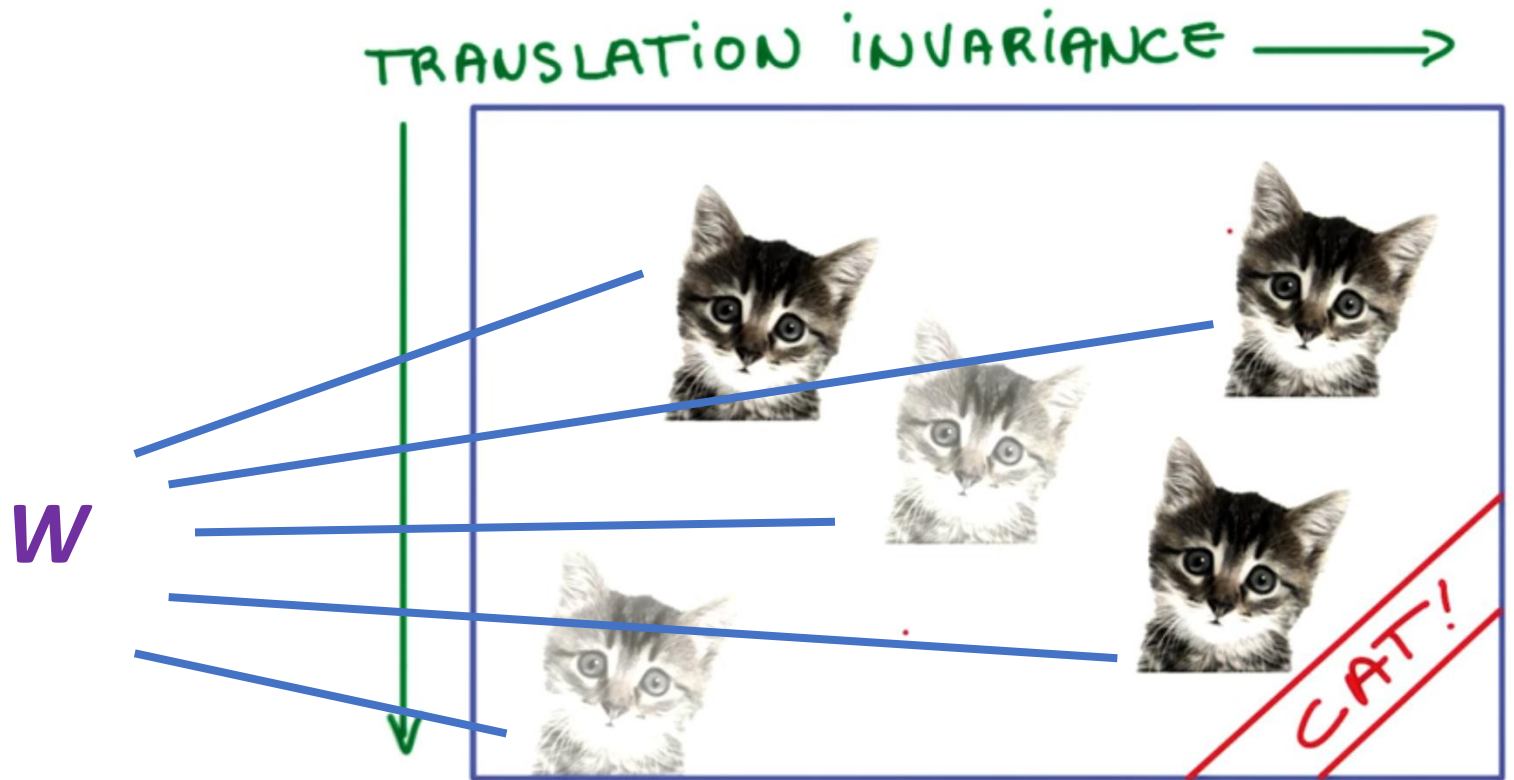
Big Picture



Why Convolution is useful?

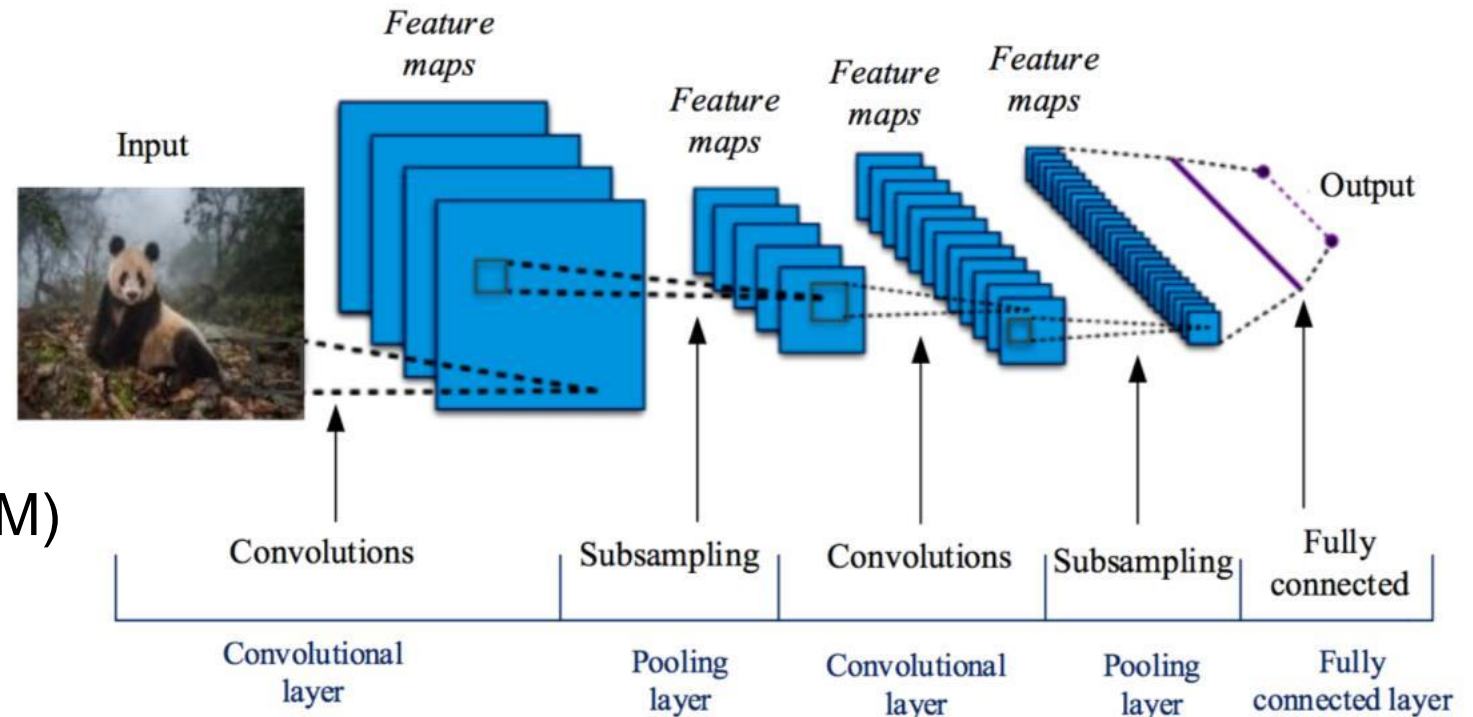
- **Idea of convolution:** weight sharing:
 - The cat is cat, It does not matter where the cat is in the image!

Sharing the weights



Convolutional Neural Network (CNN) Architecture

- Idea: locally connected network and weight sharing
- Convolution layer (CONV)
- ReLU
- Pooling Layer
- Fully connected (FC)
- Optional: Classifier layer (e.g. SVM) is sometimes added at the end.

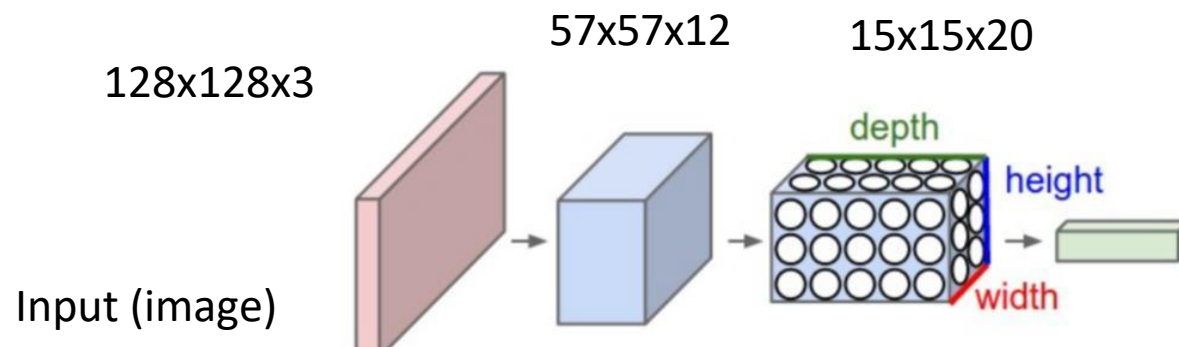
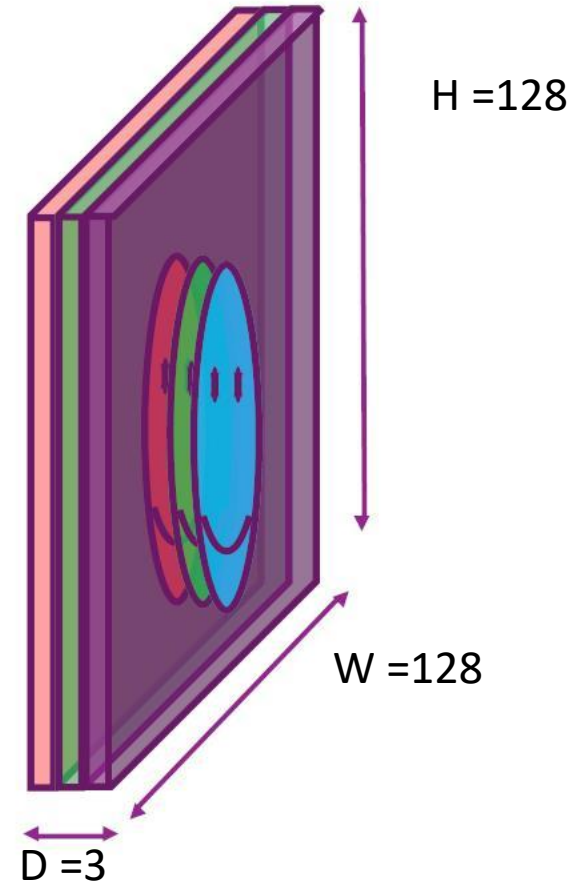


- A CNN can have all or some of these layers, but CONV layer is the must.
- Conv layer, ReLU and pooling layers can be repeated through the network.

Example of Convolution layer with image input

- **Example:** RGB image with size 128x128:
- **Goal:** classify the face expression (4 classes):
 - Class1: angry, Class2: happy, Class3: sad, Class 4:scared
- **Traditional NN:** input is represented as a vector of size $3 \times 128 \times 128 = 49152$
- **Convolutional NN:** input is represented as a 3 layers (3D):
 - Input size: 128 (W)X 128(H) X3 (D)

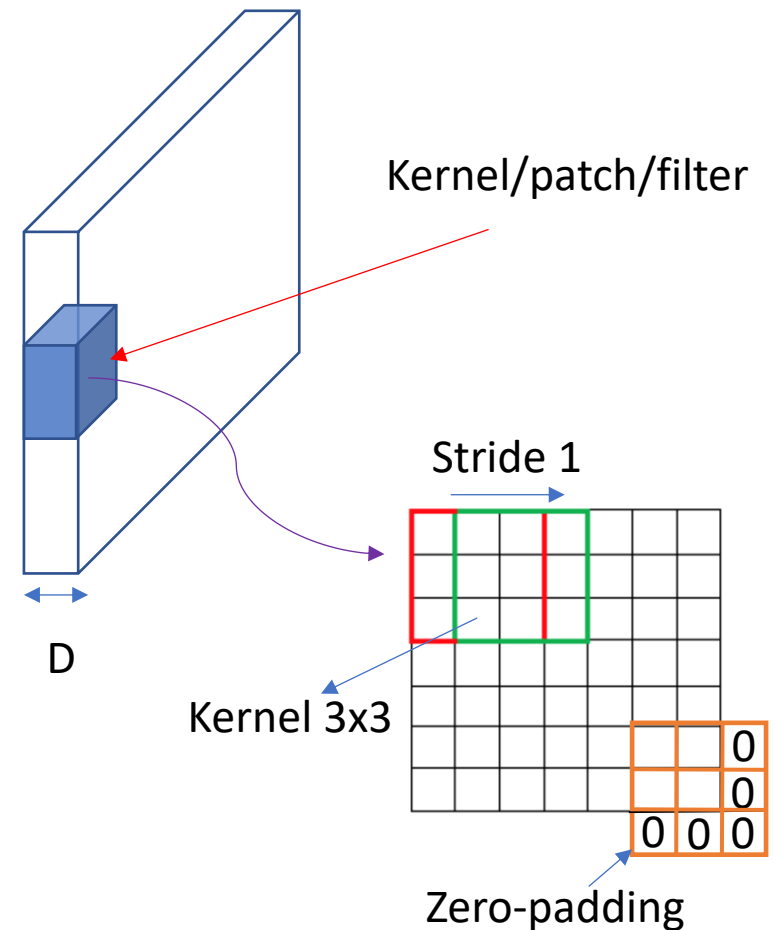
Input (RGB image)



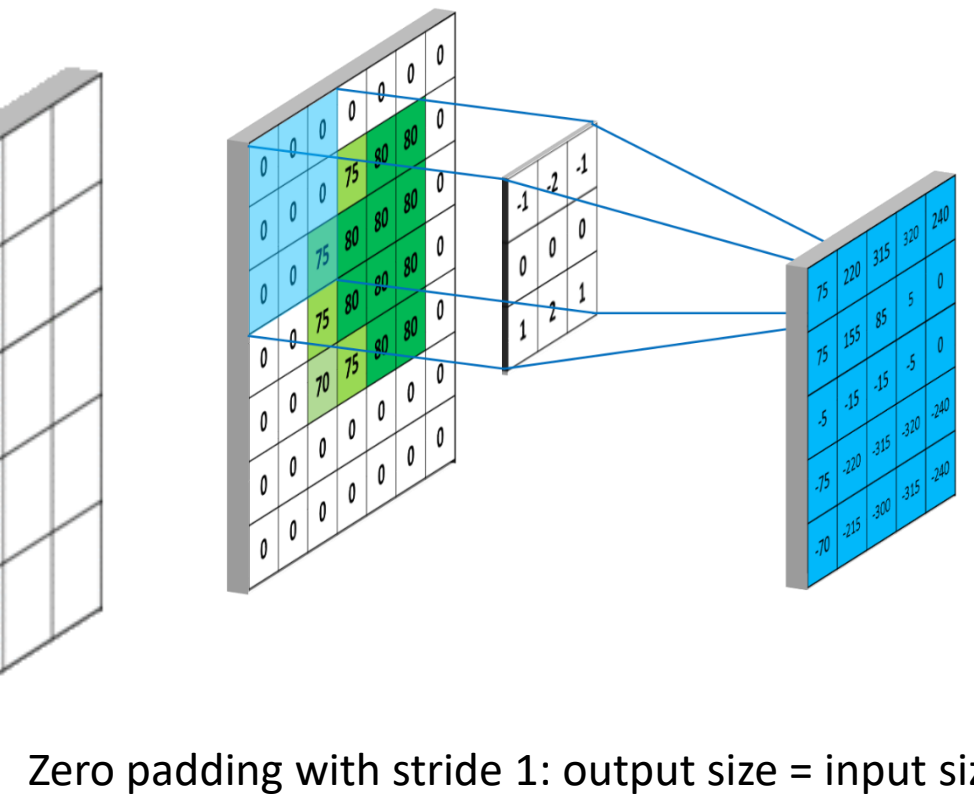
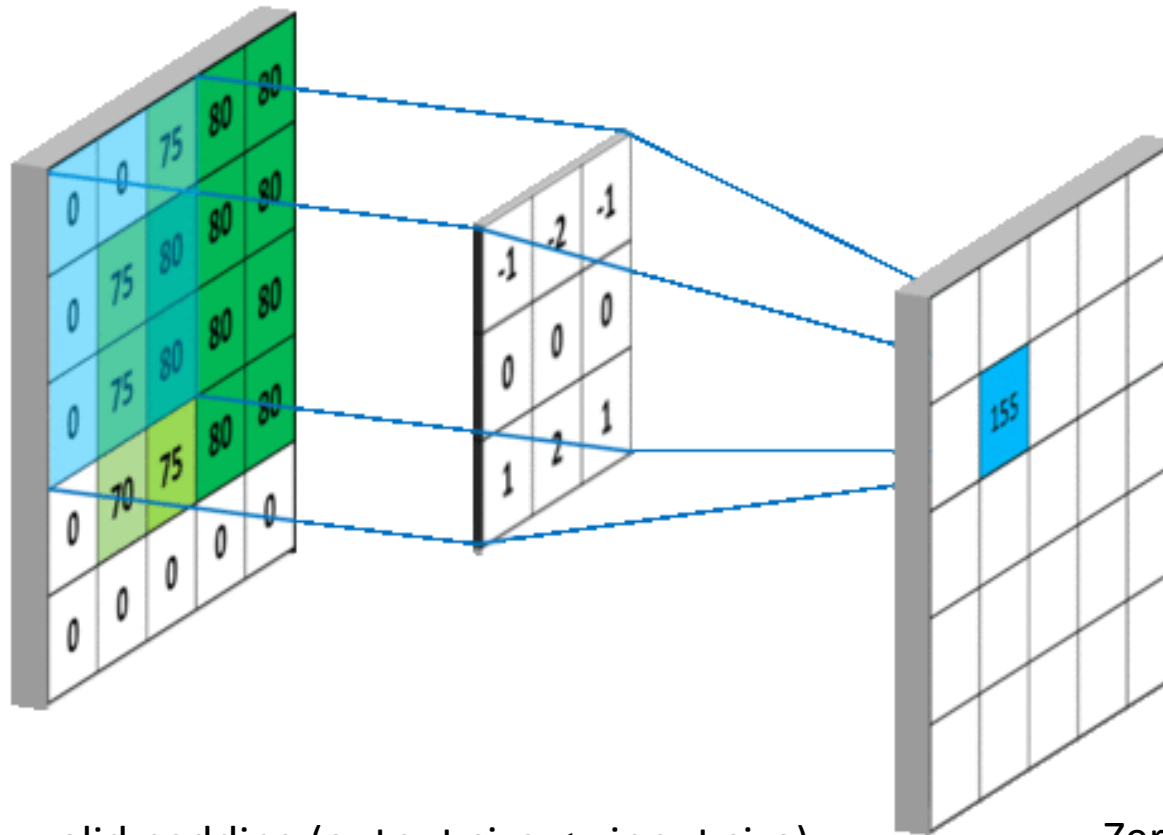
put: [0.01, **0.86**, 0.17, 0.12]

Some definitions in CNN

- **Locally connected:** only a slice of image (called patch/kernel/filter) is taken as an input. Note: the depth of the slice is the same as the depth of the layer
- **The filter/kernel size:** it is usually an odd number and it specifies the width and height of the patch
- **Stride:** it determines how many of input cells the filter shifts at a time
- **Same padding or zero-padding:** the filter covers the entire input layer. It adds zero cells for the cells beyond the image border.
- **Valid padding:** the filter never goes beyond the input cells



Convolution Example: zero padding vs valid padding



Example of Strided Convolution

3 ³	0 ⁴	1 ⁴	2	7	4
1 ¹	5 ⁰	8 ²	9	3	1
2 ⁻¹	7 ⁰	2 ³	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

$n \times n$ * $f \times f$
Padding 0 stride 2

Input

Kernel

3	4	4
1	0	2
-1	0	3

Filter size= 3x3
Padding = 0

$n \times n$ image $f \times f$ filter

padding p stride s

Output Size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

3*3+4*1+1+ 2*8- 1*2+3*2=34	

Output

Example of Strided Convolution

3	0	1 ³	2 ⁴	7 ⁴	4
1	5	8 ¹	9 ⁰	3 ²	1
2	7	2 ⁻¹	5 ⁰	1 ³	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

$n \times n$ * $f \times f$
 Padding 0 stride 2

Input

Kernel

3	4	4
1	0	2
-1	0	3

Filter size= 3x3
 Padding = 0

$n \times n$ image $f \times f$ filter

padding p stride s

Output Size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

34	3+8+28+ 8+6-2+3 = 54

Output

Example of Strided Convolution

3	0	1	2	7	4
1	5	8	9	3	1
2 ³	7 ⁴	2 ⁴	5	1	3
0 ¹	1 ⁰	3 ²	1	7	8
4 ⁻¹	2 ⁰	1 ³	6	2	8
2	4	5	2	3	9

$n \times n$ * $f \times f$
 Padding 0 stride 2

Input

Kernel

3	4	4
1	0	2
-1	0	3

Filter size= 3x3
 Padding = 0

$n \times n$ image $f \times f$ filter

padding p stride s

Output Size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

34	54
6+28 +8+6 -4+3 = 37	

Output

Example of Strided Convolution

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2 ³	5 ⁴	1 ⁴	3
0	1	3 ¹	1 ⁰	7 ²	8
4	2	1 ⁻¹	6 ⁰	2 ³	8
2	4	5	2	3	9

$n \times n$ * $f \times f$
 Padding 0 stride 2

Input

Kernel

3	4	4
1	0	2
-1	0	3

Filter size= 3x3
 Padding = 0

$n \times n$ image $f \times f$ filter

padding p stride s

Output Size:

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

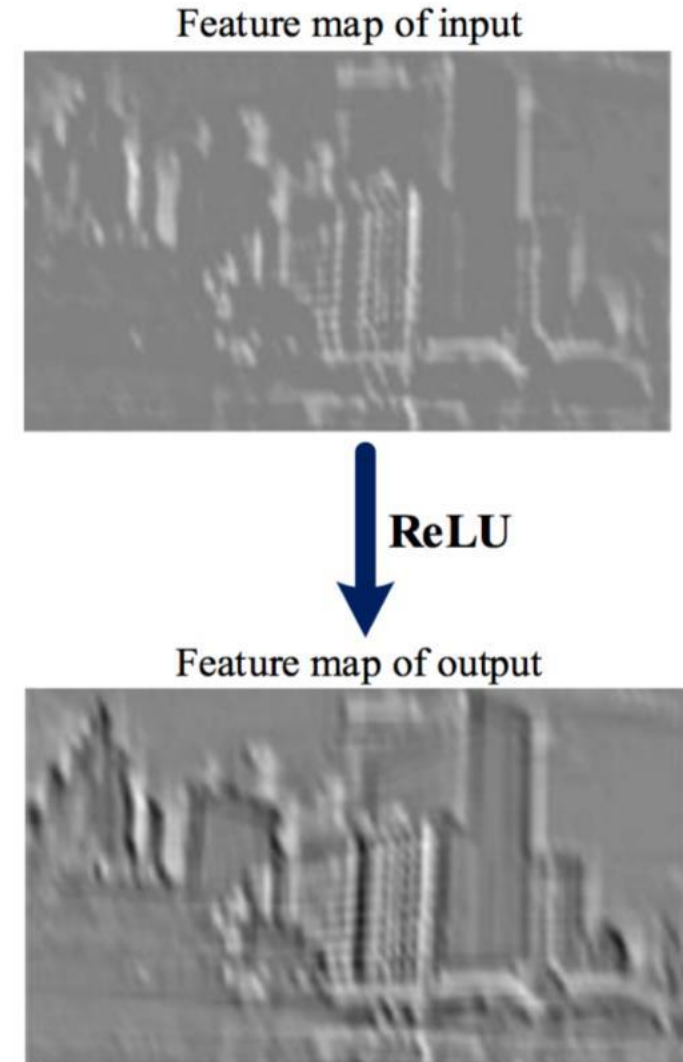
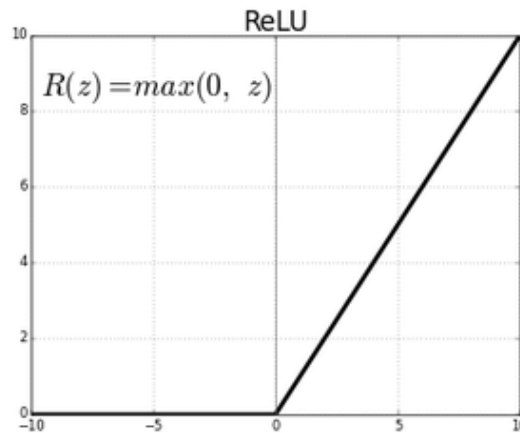
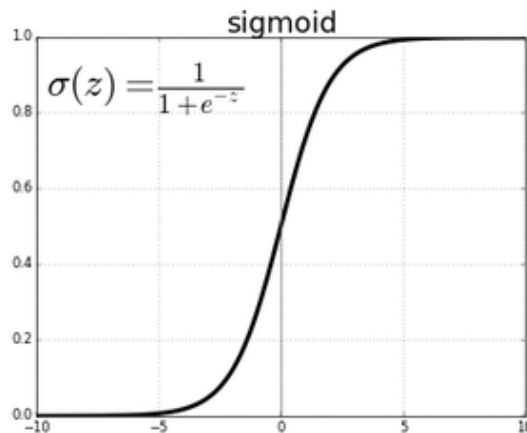
34	54
37	6+20+4+3+14- 1+6= 52

Output

34	54
37	52

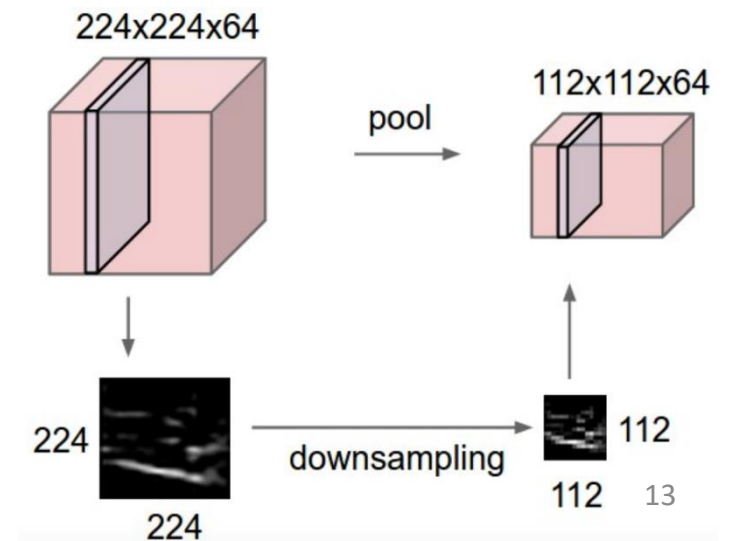
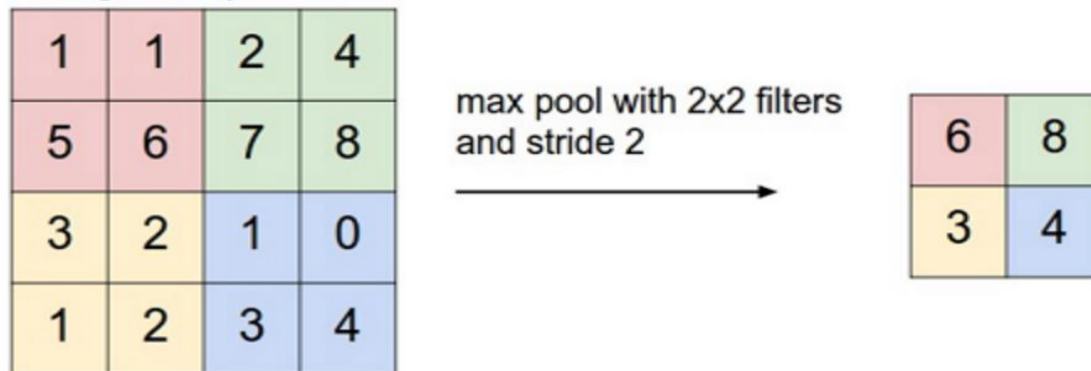
ReLU layer output example

- ReLU is usually considered as a part of convolution layer since it does not change the size of output.
Conv+ReLU
- ReLU can act as edge detection and increase the contrast in the images.
 - Idea: Values below some threshold are mapped to zero



Pooling Layer

- In many cases, the pooling layer is added after conv layer +ReLU for decreasing the size of the output and other purposes.
- No parameters are learned for pooling layer!
- Two common pooling functions is **max pooling** and **average pooling**:
 - Input cells under that patch are shrunk to a single cell
 - Pooled value is either the maximum or average of all cells under that patch.
- Similar to conv layer, pooling uses kernels/filters with specific **size** and **stride**.
 - One of the common pooling filter is 2x2 with stride 2.



Putting together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.

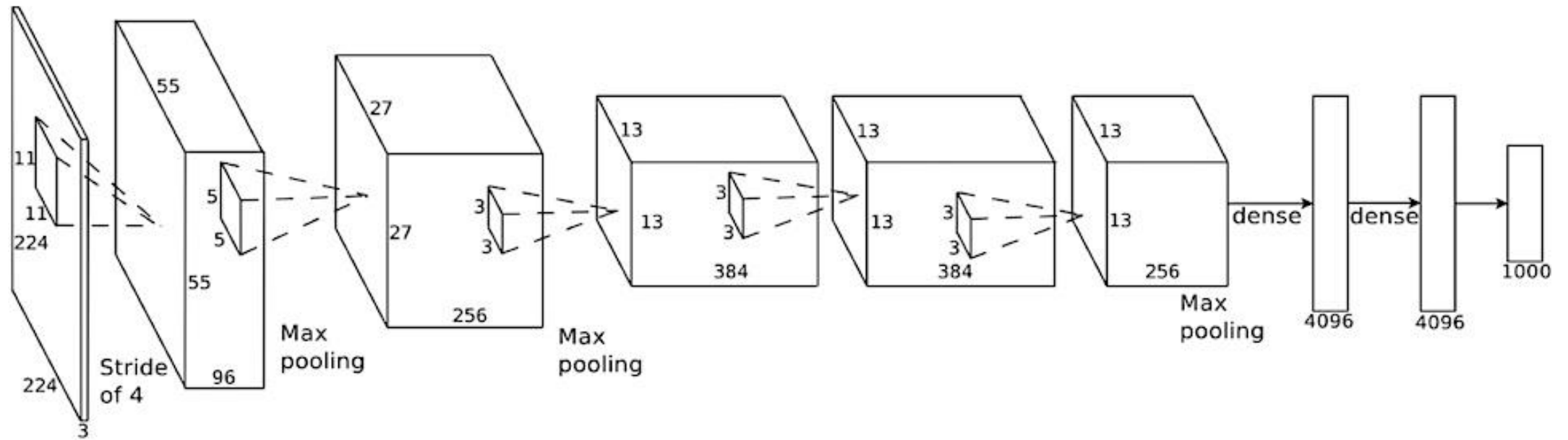


$\hat{y}$

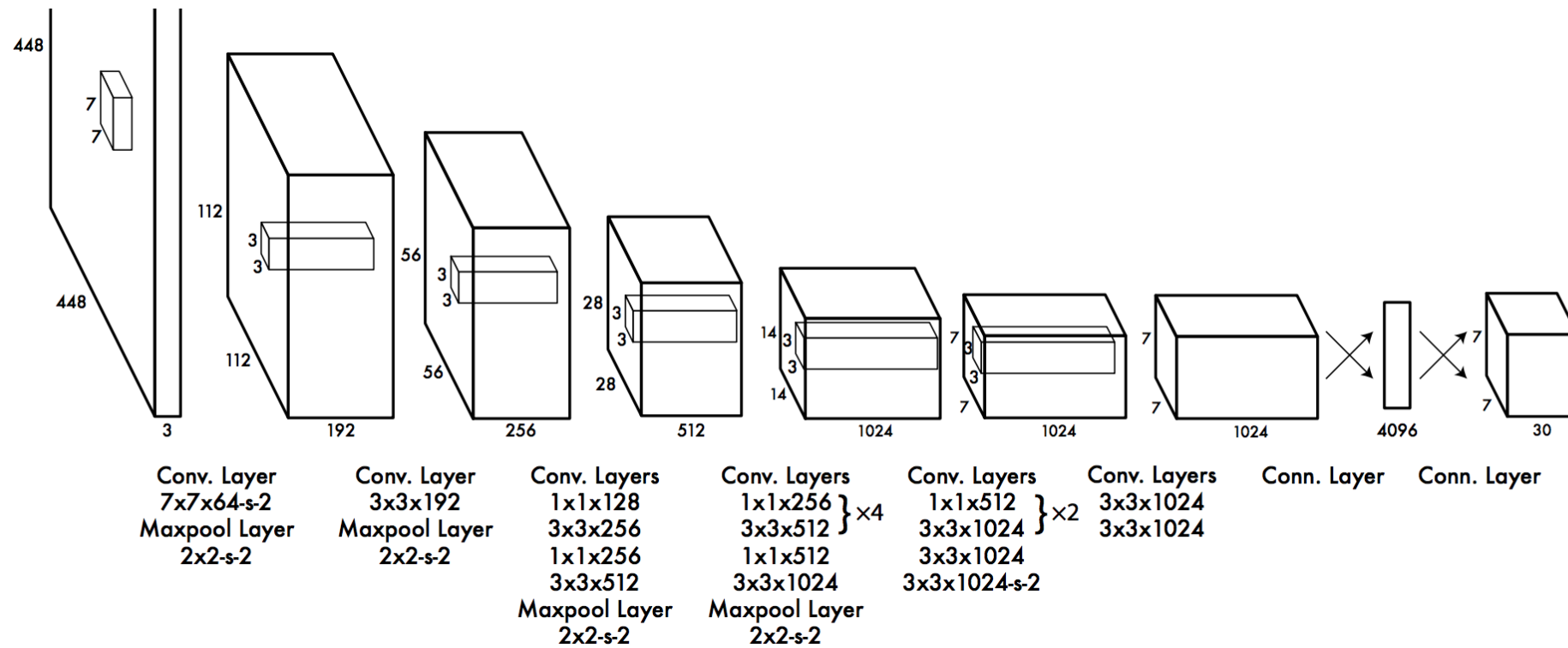
$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J

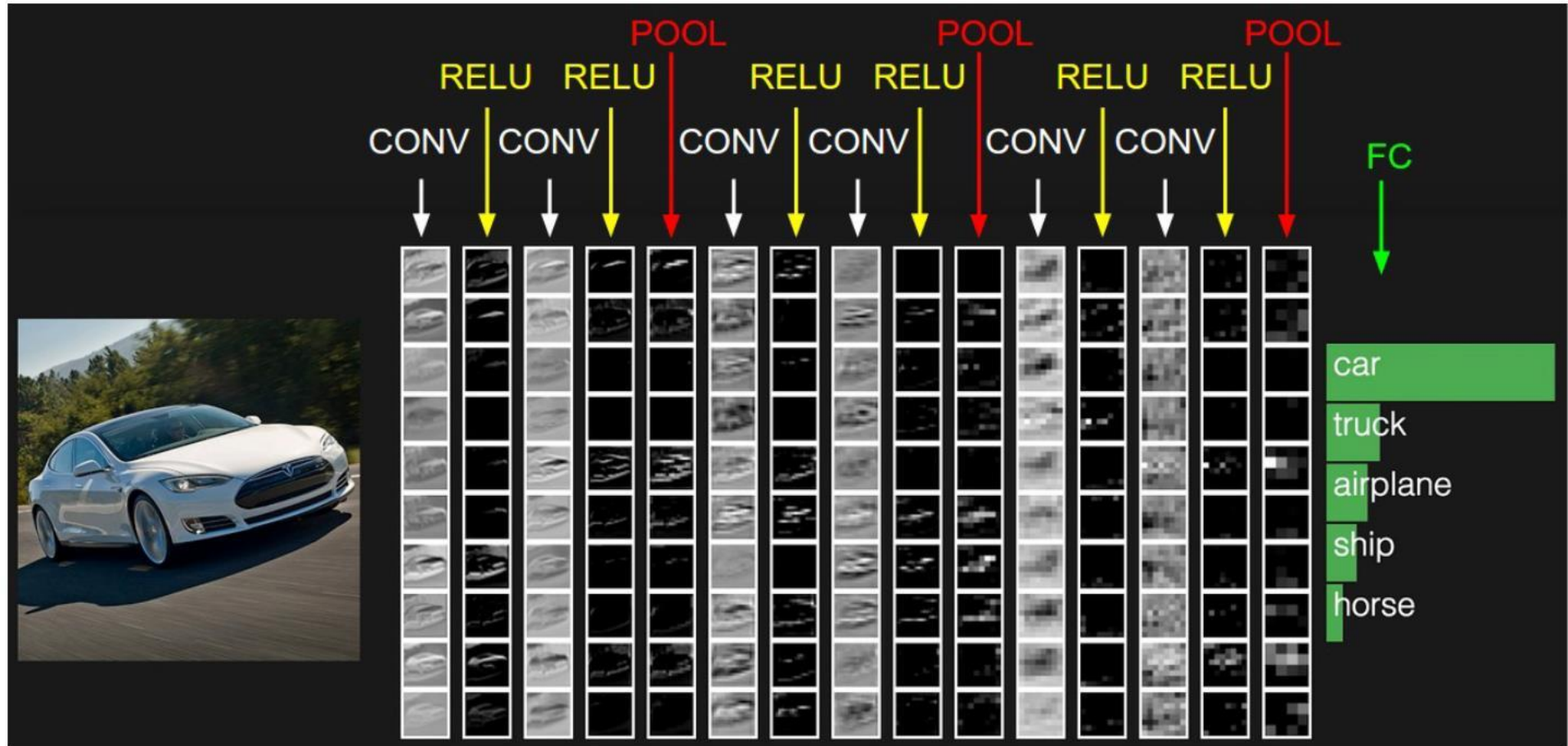
Convolutional Net Example - AlexNet



YOLO Network Architecture [2]



Putting together: CNN for classification



CNN: some computer vision applications

Semantic Segmentation



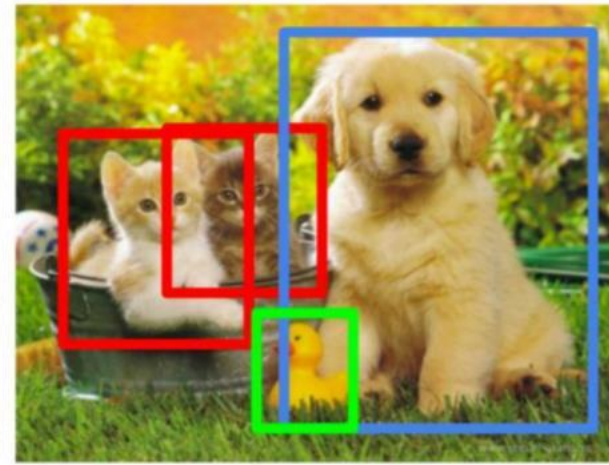
CAT

Classification + Localization



CAT

Object Detection



CAT, DOG, DUCK

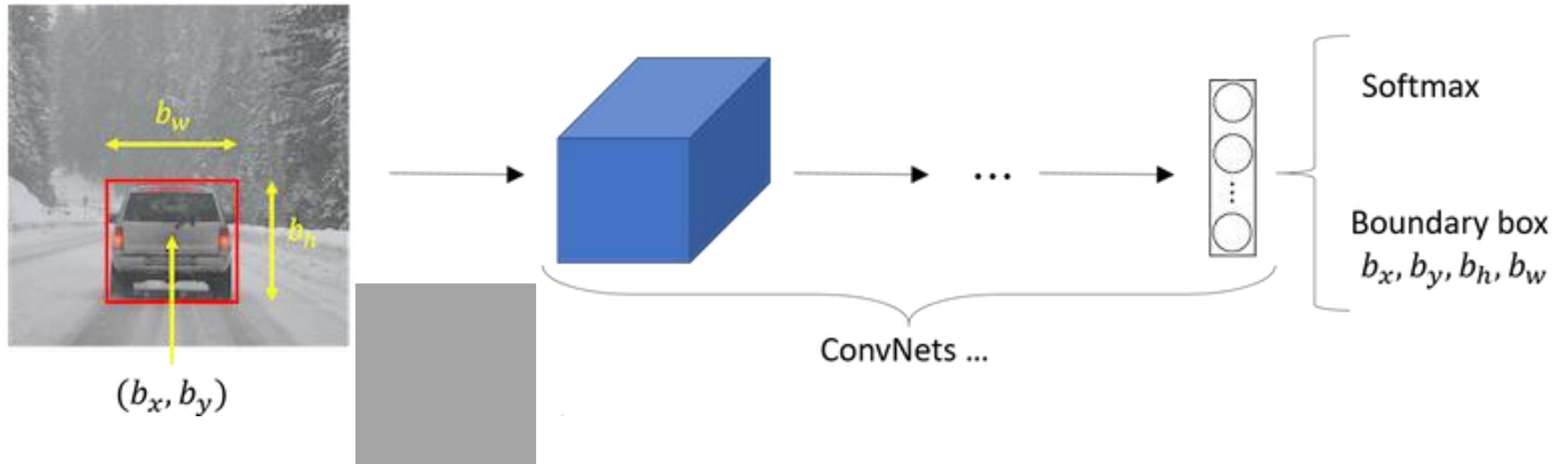
Instance Segmentation



CAT, DOG, DUCK

Backup slides

Object detection: Classification with localization



Object detection: defining target label y

- 1 - pedestrian
- 2 - car
- 3 - motorcycle
- 4 - background

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Need to output b_x, b_y, b_h, b_w , class label (1-4)



$$\mathcal{L}(\hat{y}, y) = \begin{cases} (\hat{y}_1 - y_1)^2 + \dots + (\hat{y}_8 - y_8)^2, & y_1 = 1 \\ (\hat{y}_1 - y_1)^2, & y_1 = 0 \end{cases}$$

$$\begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

Object Detection

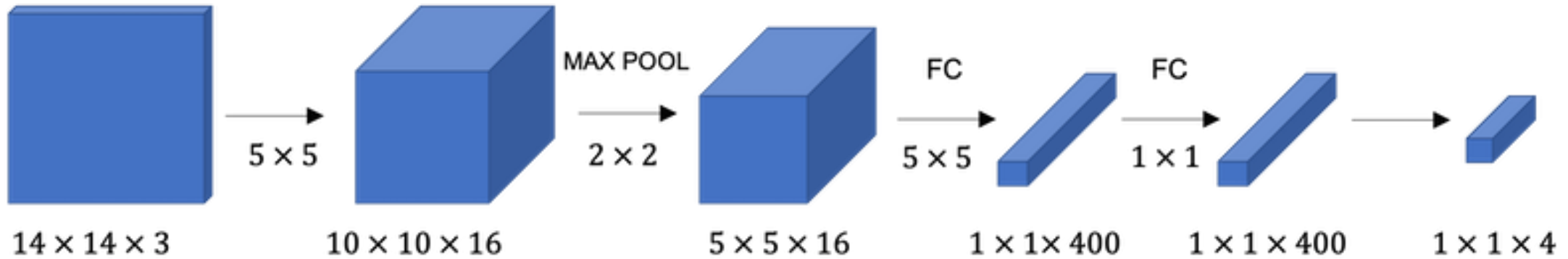
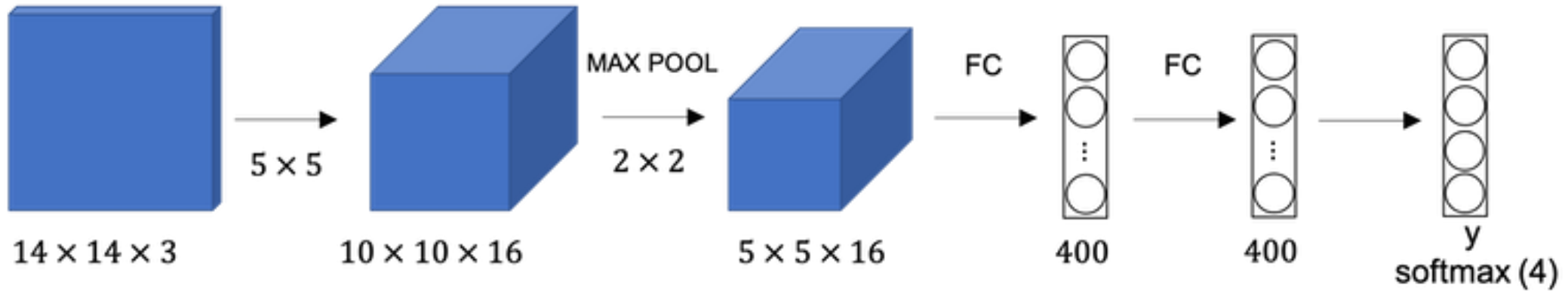
Sliding window

Sliding window until it finds the object

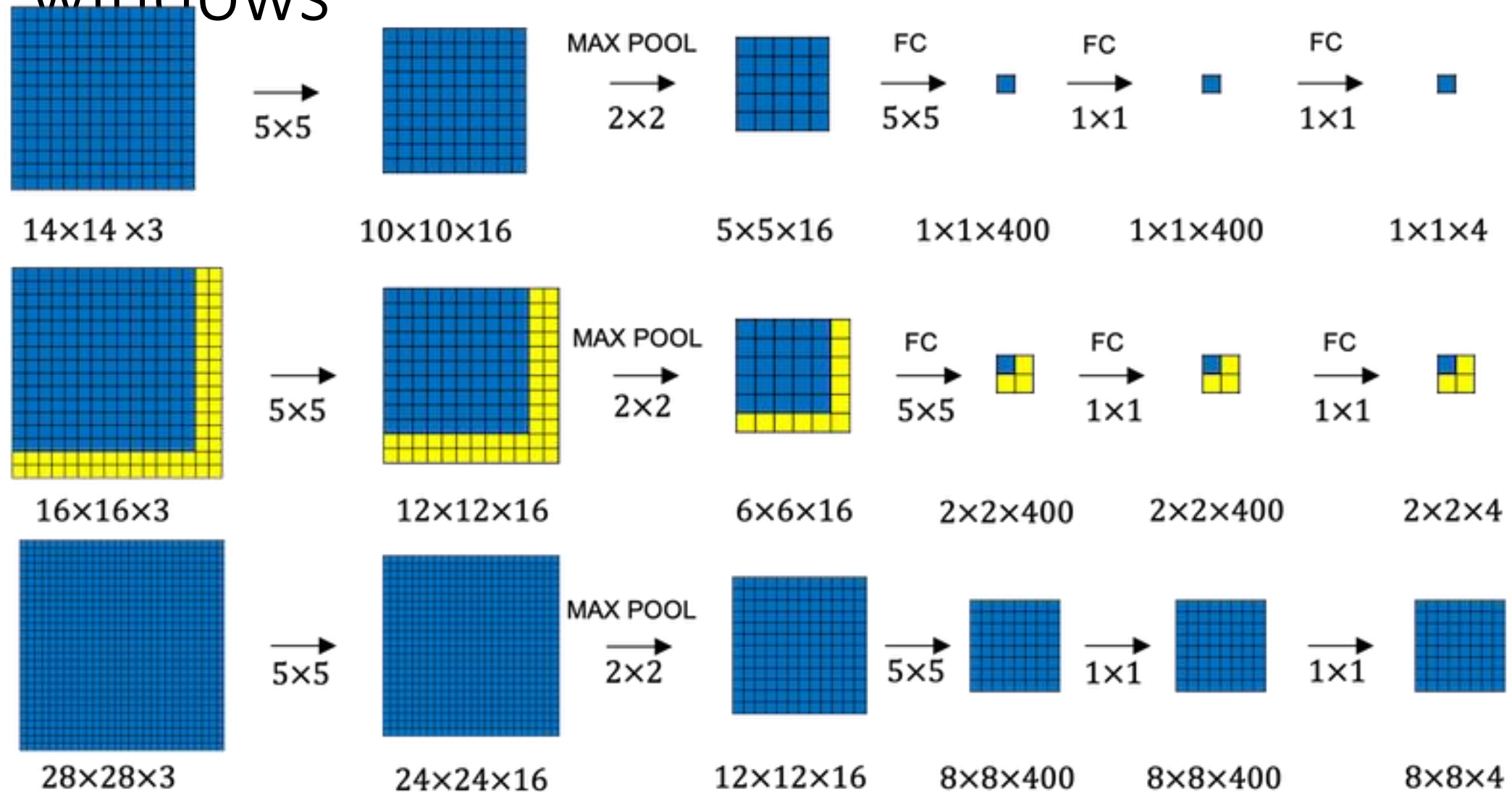


<https://www.pyimagesearch.com/2015/03/23/sliding-windows-for-object-detection-with-python-and-opencv/>

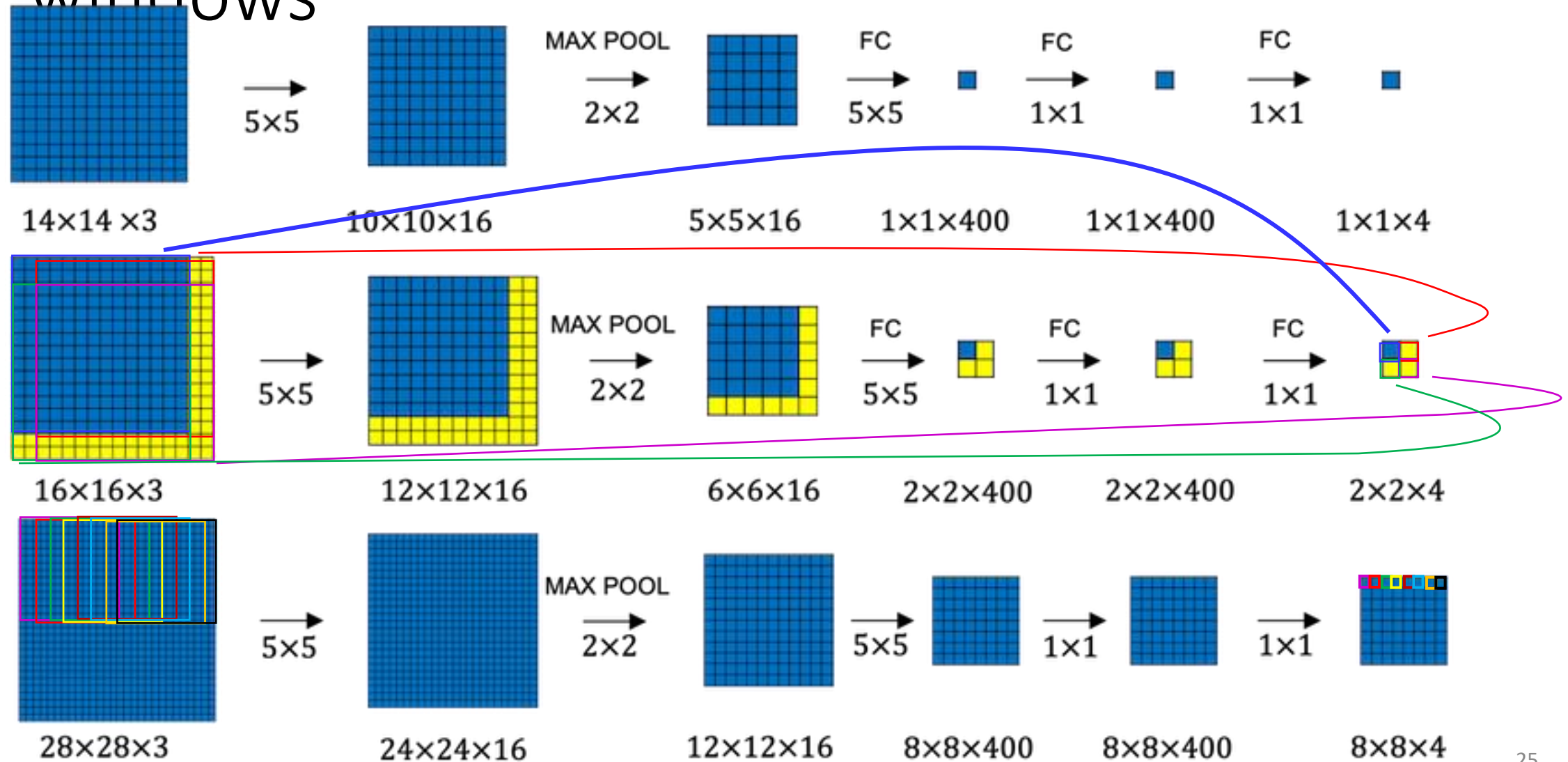
Turning FC layers to convolutional layers



Convolution Implementation of Sliding windows



Convolution Implementation of Sliding windows



Object Detection

Sliding window

Sliding window until it finds the object

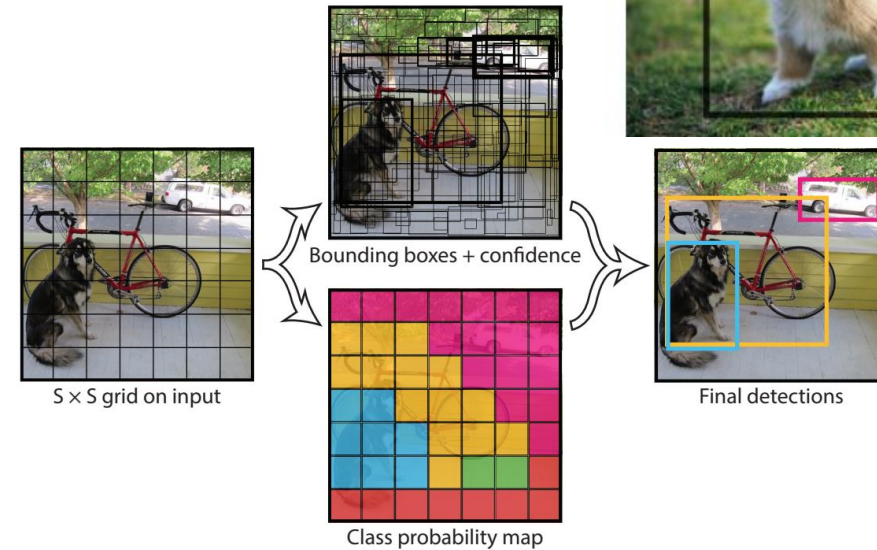
Issue: bbox size

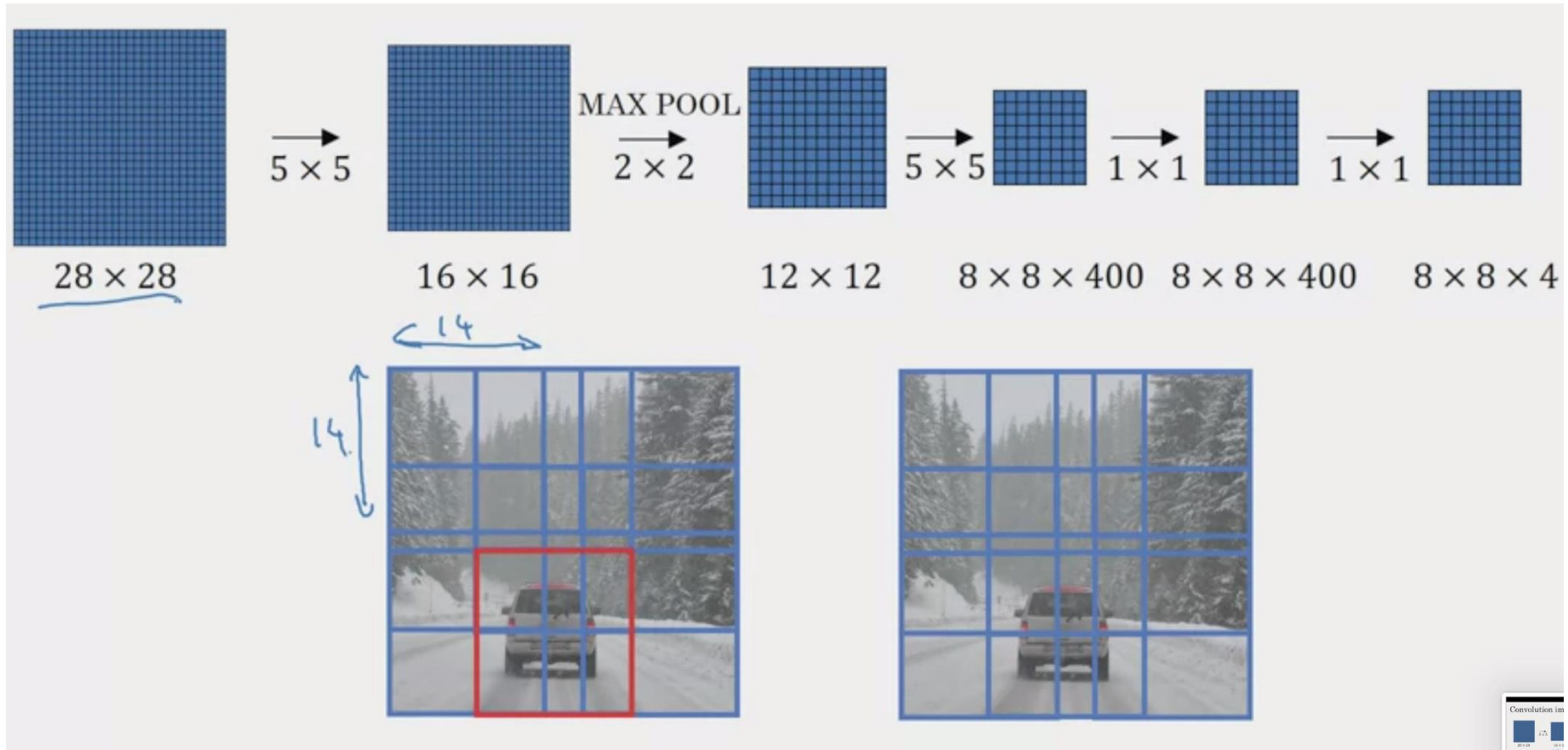


<https://www.pyimagesearch.com/2015/03/23/sliding-windows-for-object-detection-with-python-and-opencv/>

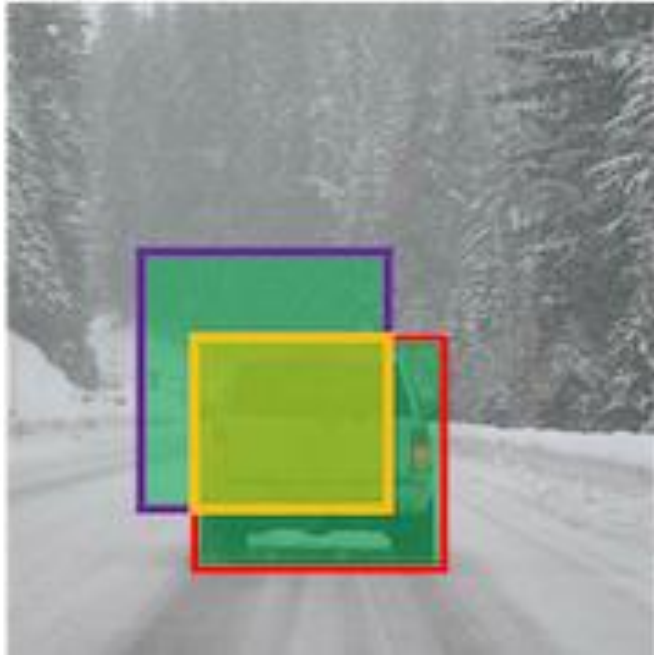
YOLO[2]:

Propose the bounding box and the detected object simultaneously





Evaluating object localization: Intersection over Union



Intersection Over Union (IOU)

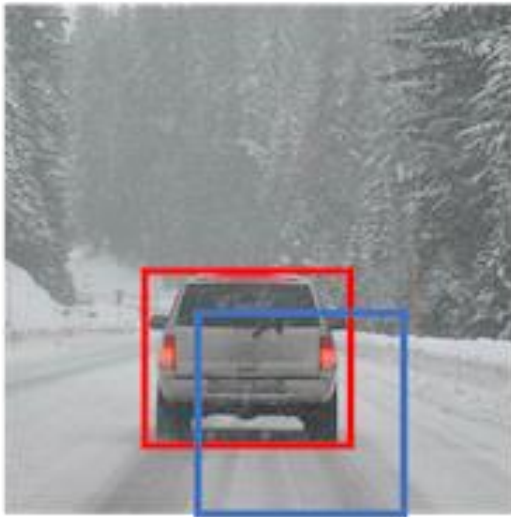
$$= \frac{\text{size of orange area}}{\text{size of green area}}$$

Correct if $IOU \geq 0.5$

The purple box is the output bounding box
The red one is the ground-truth bounding box

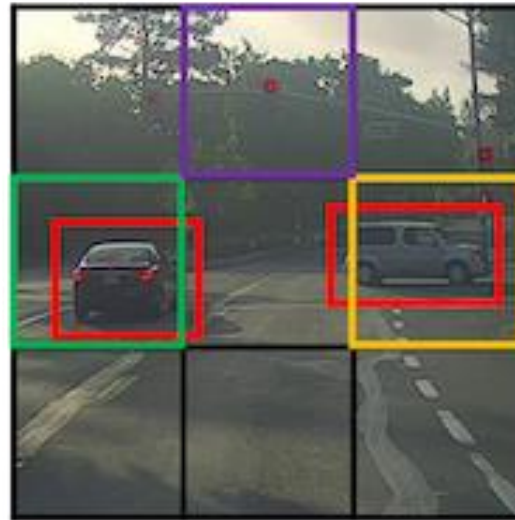
YOLO algorithm

Problem:



None of sliding windows match up perfectly with the position of the car

YOLO algorithm (You Only Look Once)



Input: 100 x 100
Grid: 3 x 3

Labels for training for each grid cell:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} \quad \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

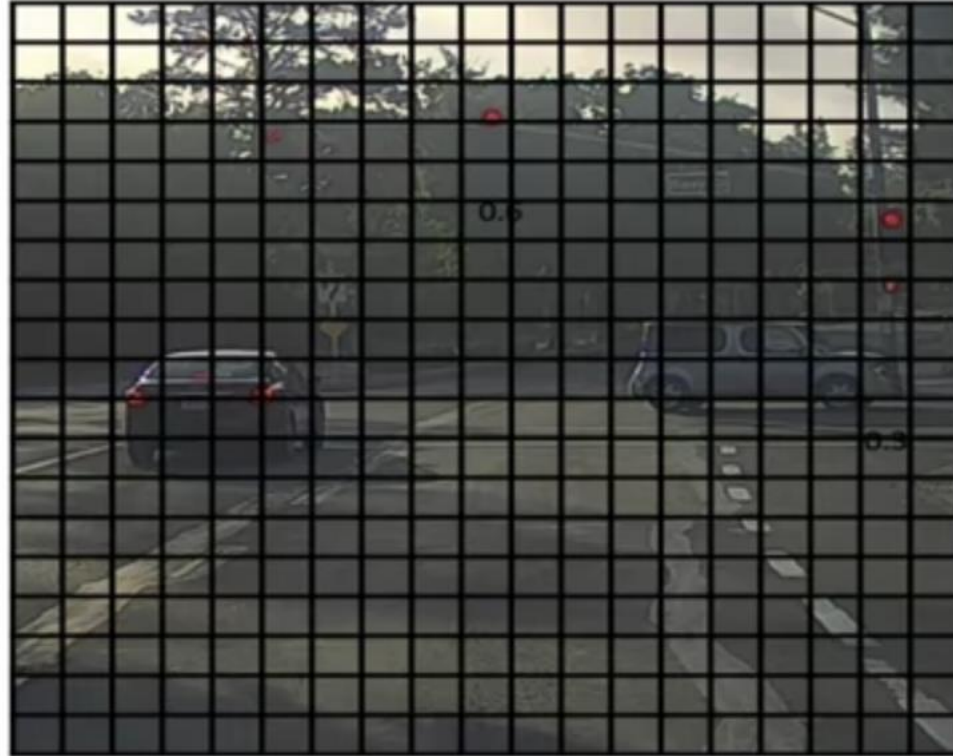
Target output: 3 x 3 x 8



Non-max suppression Example



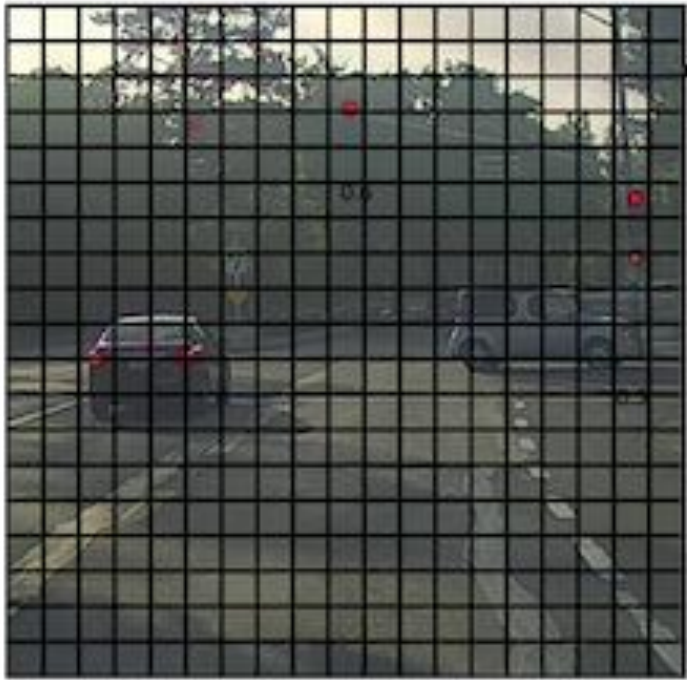
Non-max suppression Example



Non-max suppression Example: multiple Detection



Non-max suppression Algorithm



19×19

Each output prediction is:

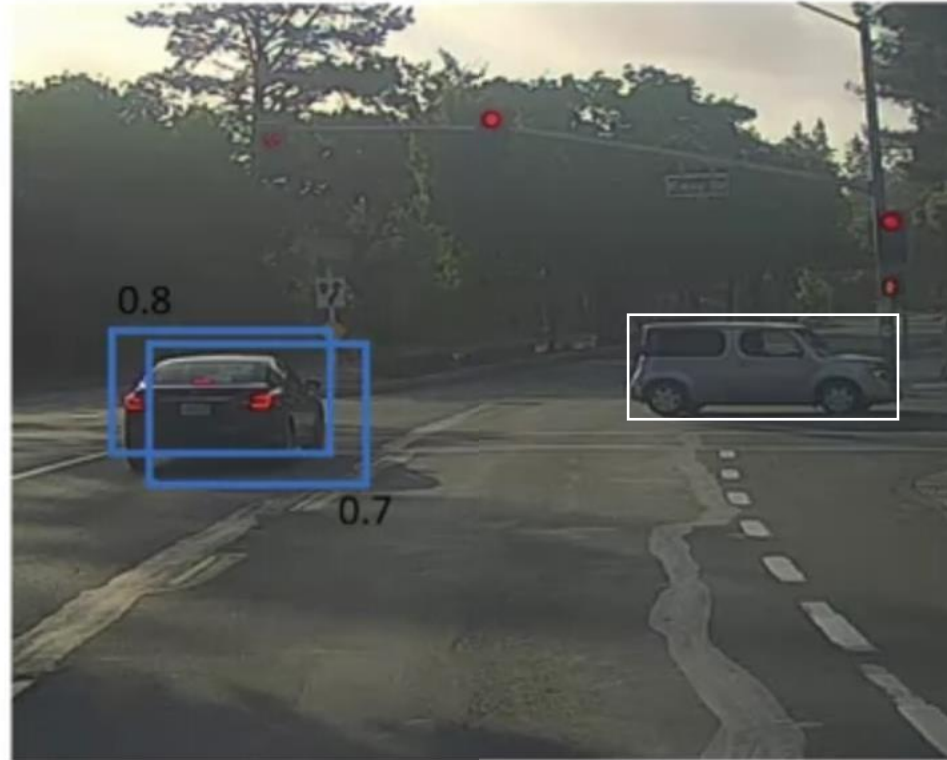
$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \end{bmatrix}$$

Discard all boxes with $p_c \leq 0.6$

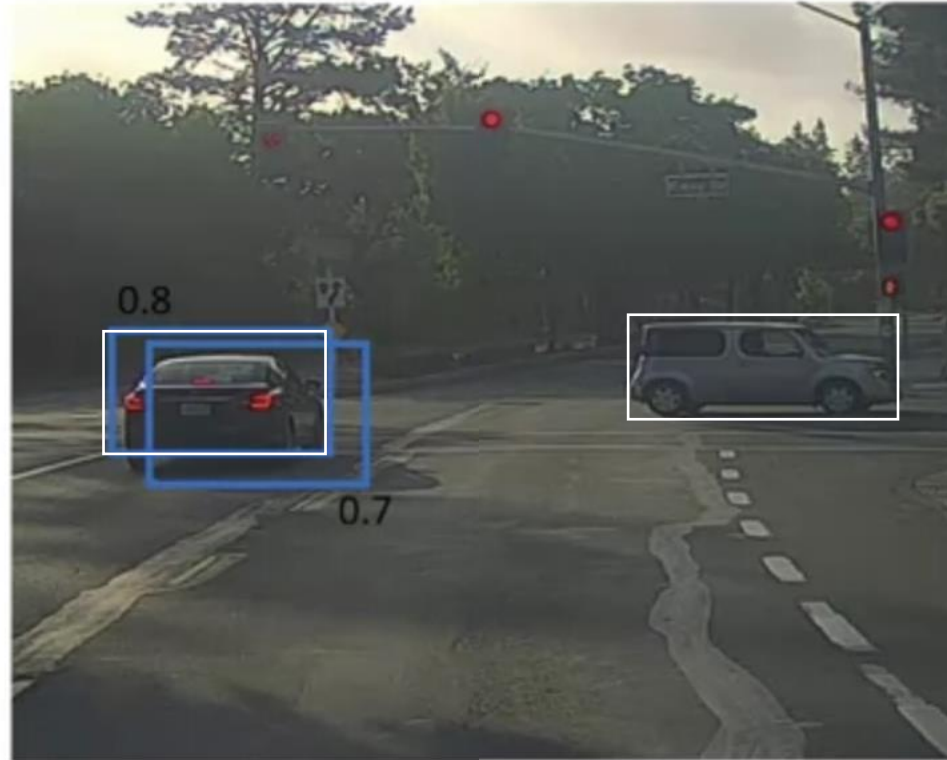
While there are any remaining boxes:

- Pick the box with the largest p_c . Output that as a prediction.
- Discard any remaining box with $\text{IOU} \geq 0.5$ with the box output in the previous step

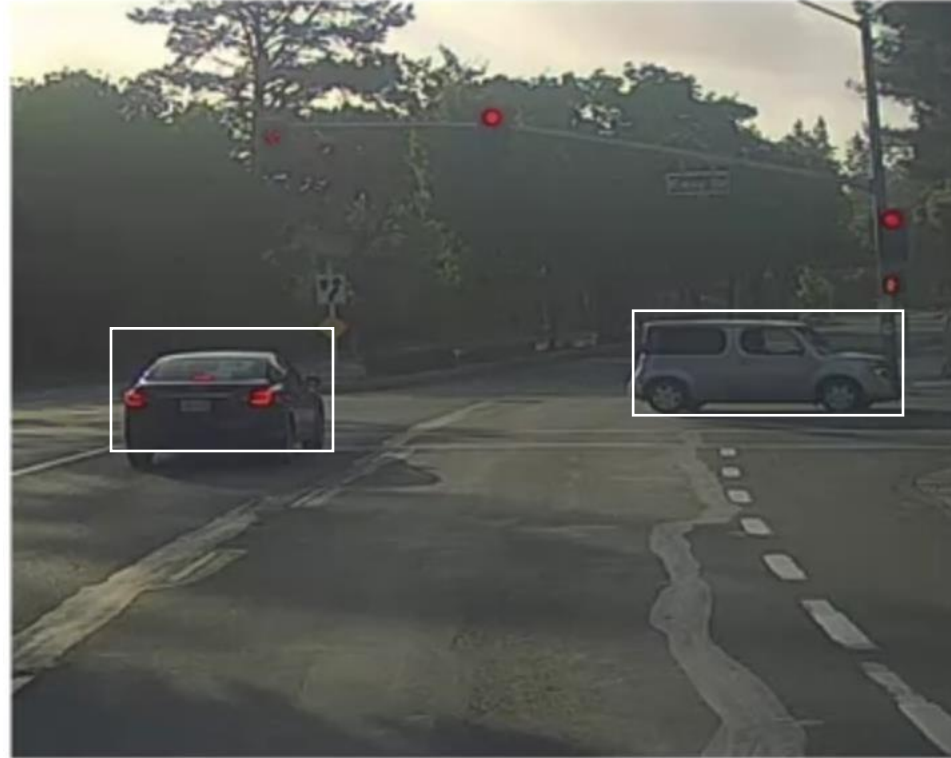
Non-max suppression Example: multiple Detection



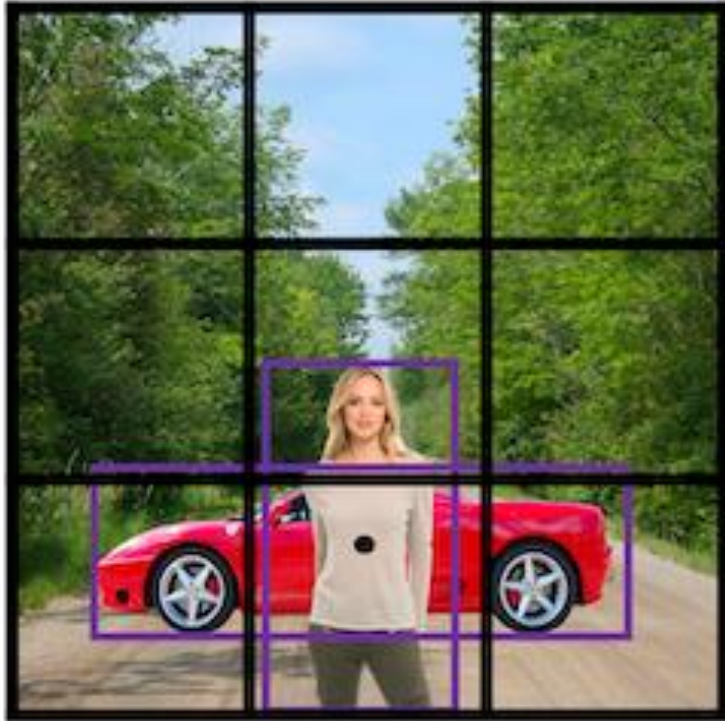
Non-max suppression Example: multiple Detection



Non-max suppression Example



Overlapping boxes and Anchor box Example



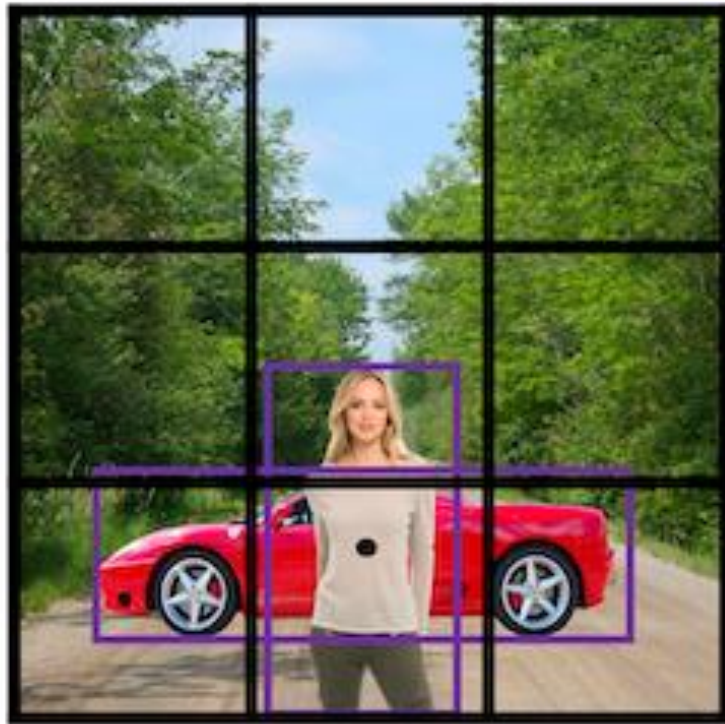
Anchor box 1:



Anchor box 2:



Overlapping boxes and Anchor box Example



Anchor box 1:



Anchor box 2:



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 1 \\ 0 \\ 0 \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Car only?

$$\begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Anchor box 1

Anchor box 2

YOLO Algorithm -training

1 - pedestrian

2 - car

3 - motorcycle

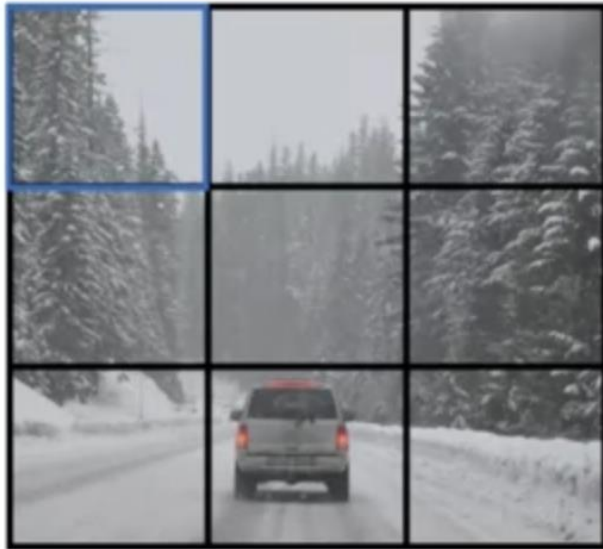


$y =$

$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

y is $3 \times 3 \times 2 \times 8$

YOLO Algorithm -training



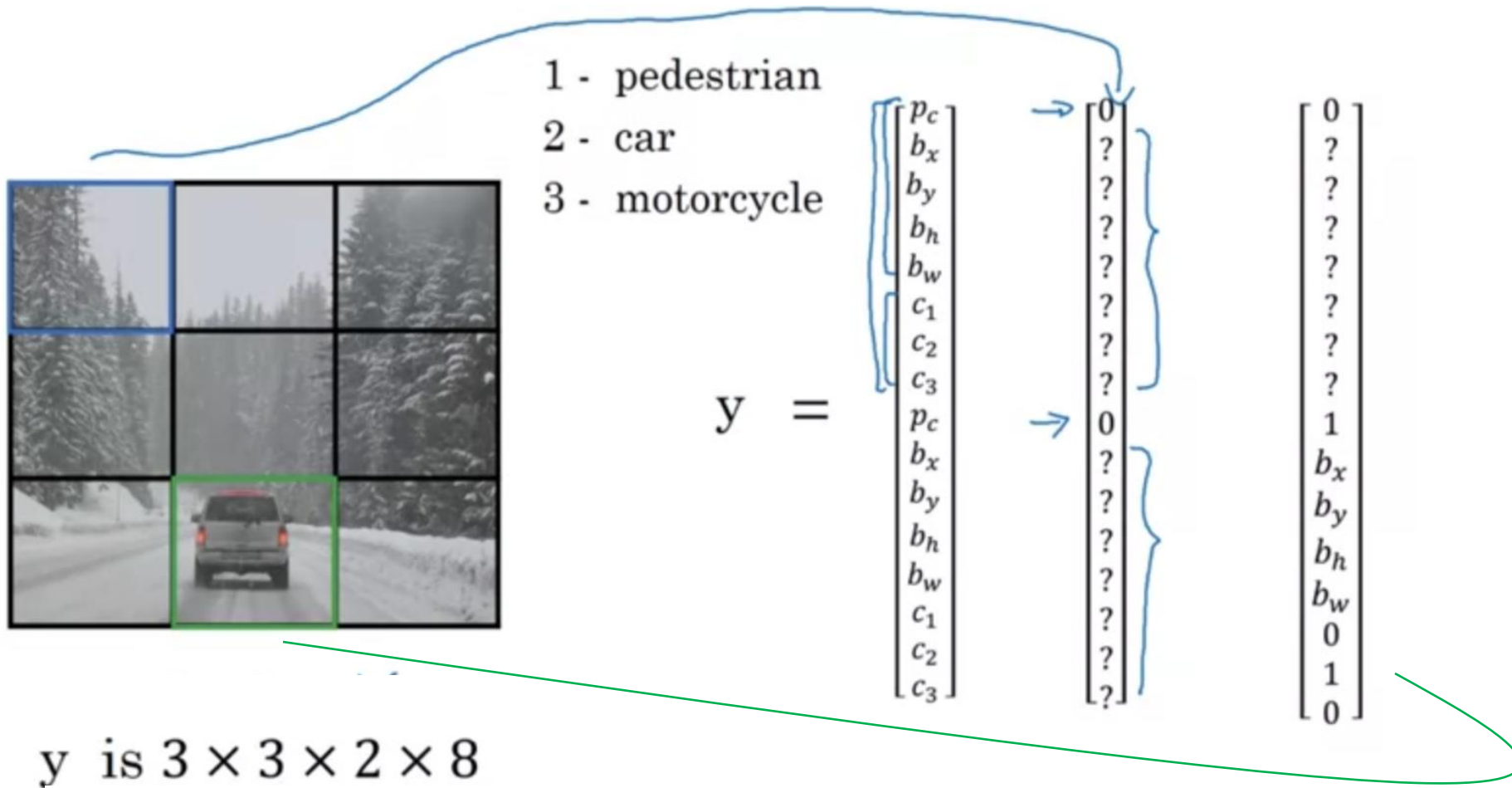
- 1 - pedestrian
- 2 - car
- 3 - motorcycle

$y =$

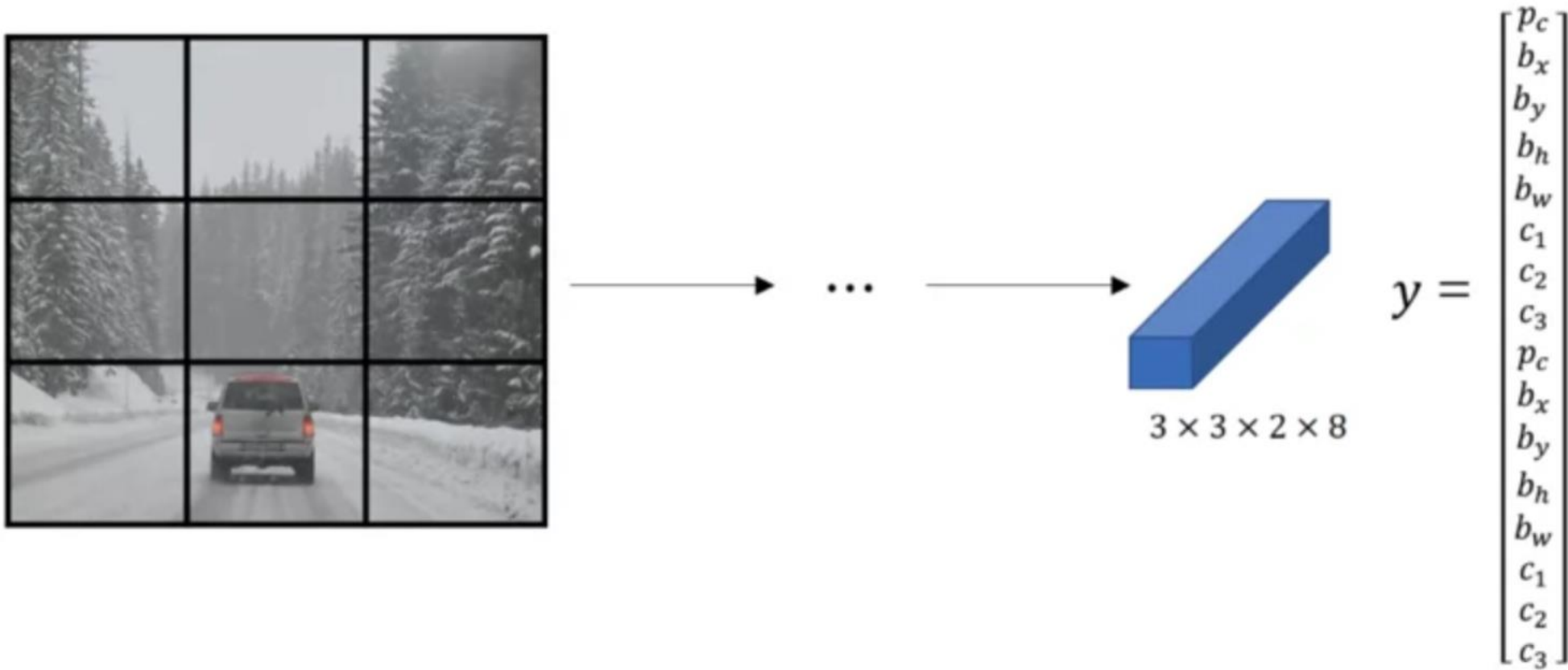
$$\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}
 \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix}$$

y is $3 \times 3 \times 2 \times 8$

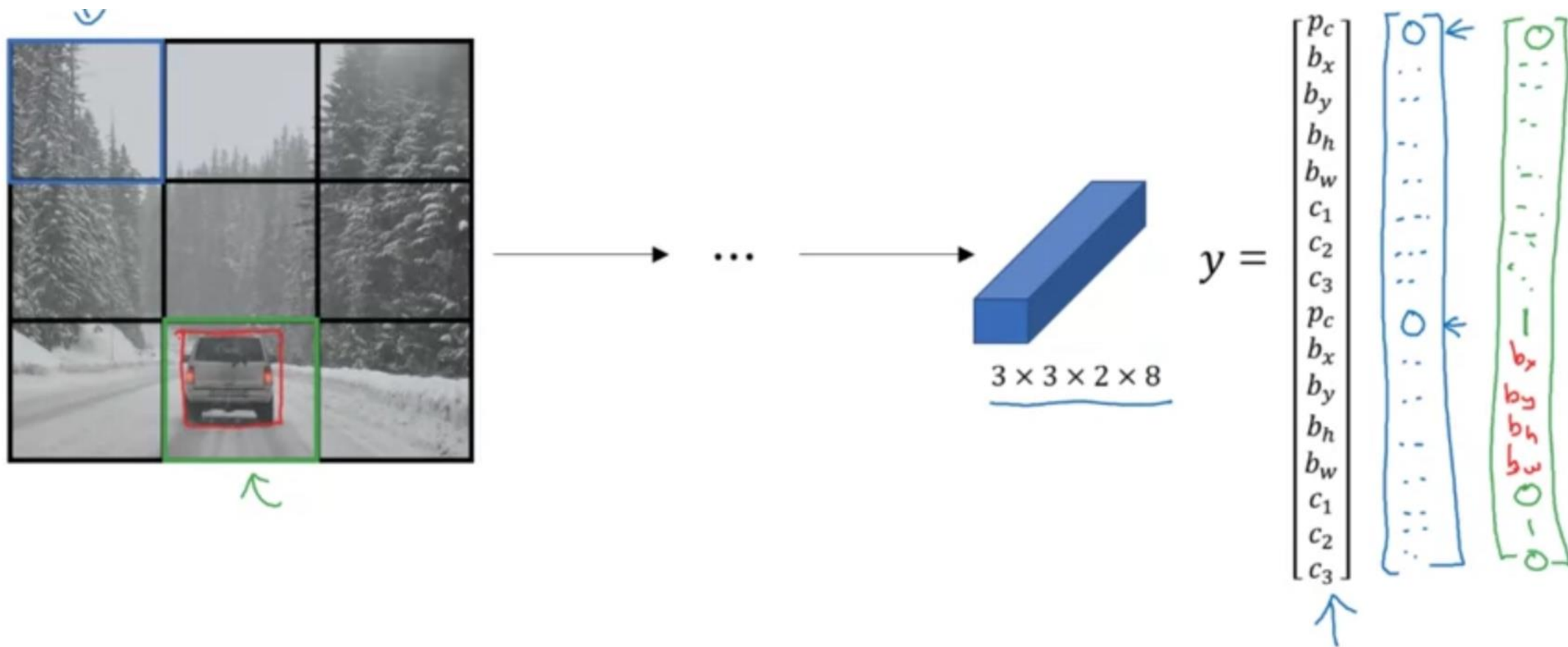
YOLO Algorithm -training



Making prediction



Making prediction



Outputting the non max suppressed outputs

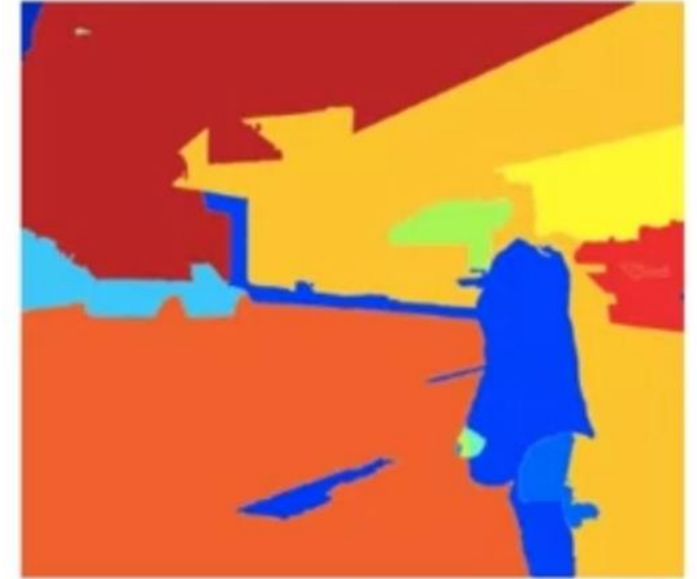


Outputting the non max suppressed outputs



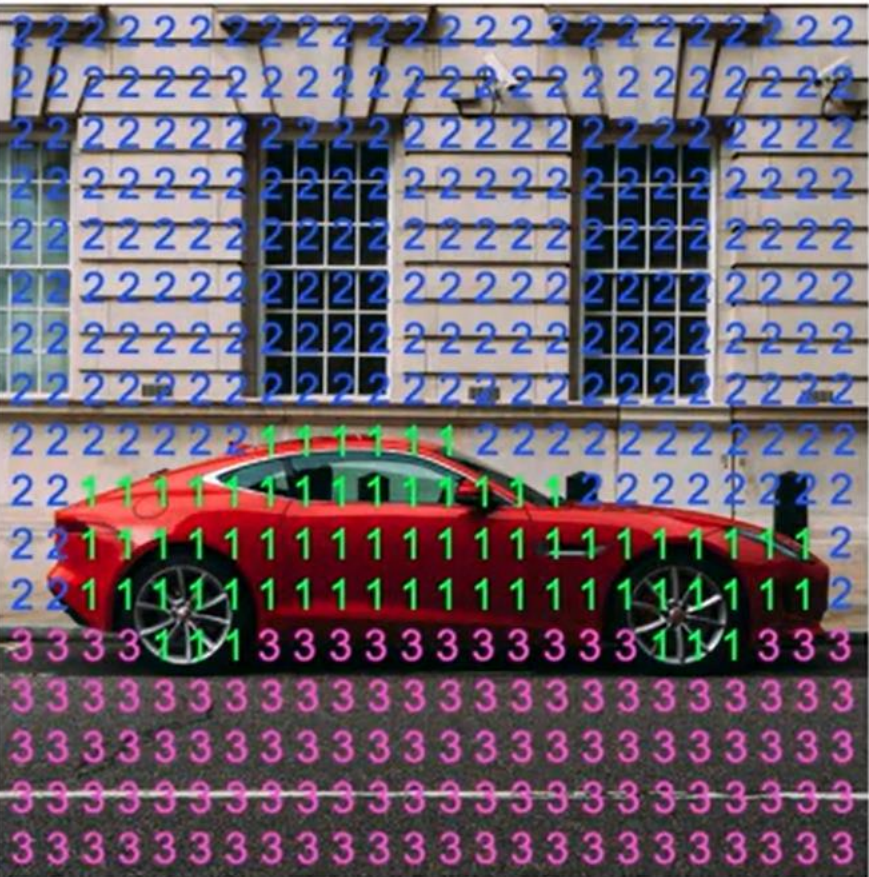
1. If you're using two anchor boxes, then for each of the nine grid cells, you get two predicted bounding boxes.
2. Next, you then get rid of the low probability predictions.
3. And then finally if you have three classes you're trying to detect, you're trying to detect pedestrians, cars and motorcycles. What you do is, for each of the three classes, independently run non-max suppression for the objects that were predicted to come from that class.

R-CNN: Regional Proposal

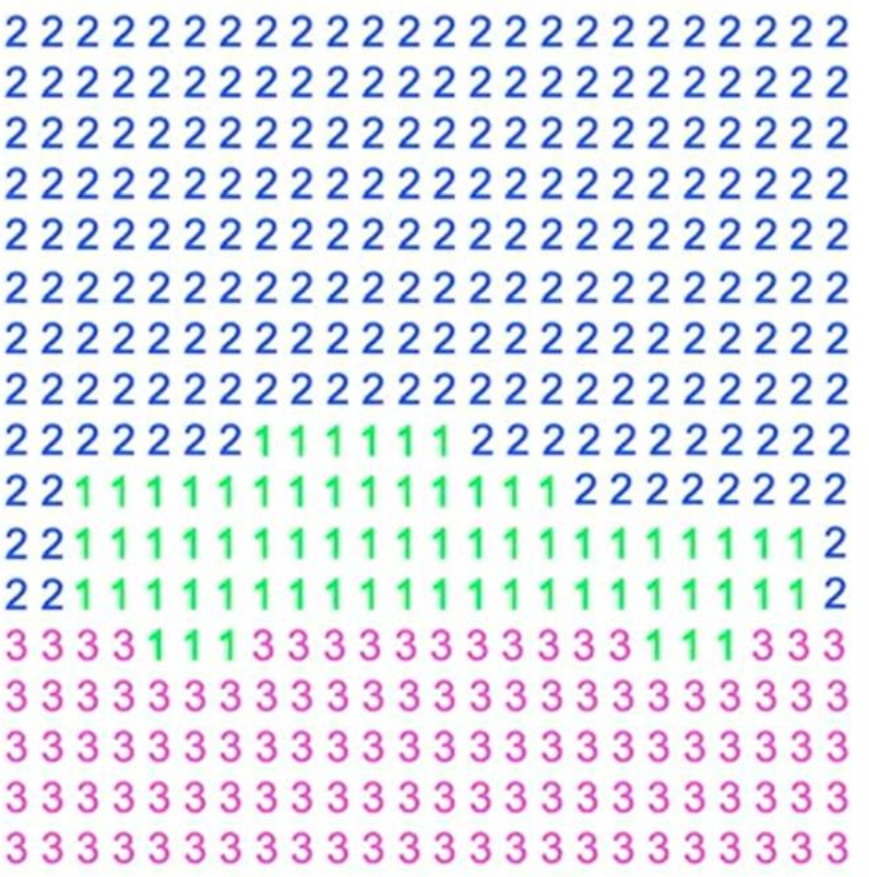


algorithm	description
R-CNN	Propose regions. Classify proposed regions one at a time. Output label + bounding box. The way that they perform the region proposals is to run an algorithm called a segmentation algorithm. One downside of the R-CNN algorithm was that it is actually quite slow.
Fast R-CNN	Propose regions. Use convolution implementation of sliding windows to classify all the proposed regions. One of the problems of fast R-CNN algorithm is that the clustering step to propose the regions is still quite slow.
Faster R-CNN	Use convolutional network to propose regions. (Most implementations are usually still quit a bit slower than the YOLO algorithm.)

Semantic Segmentation

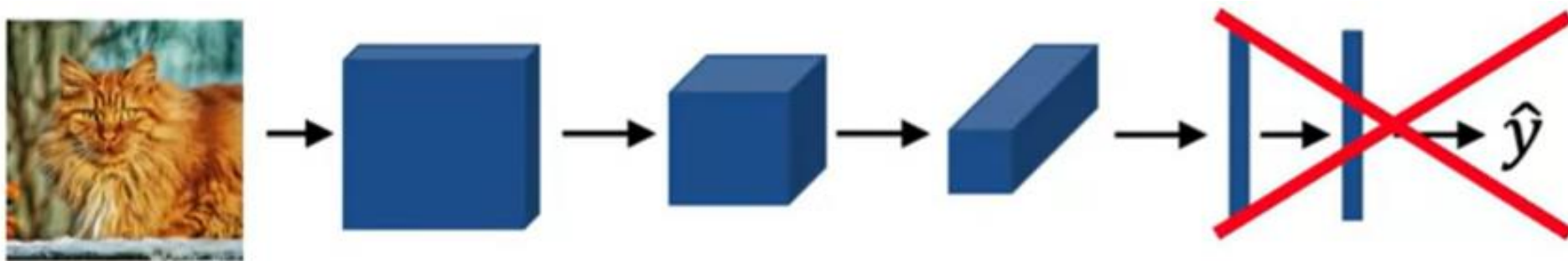


- 1. Car
- 2. Building
- 3. Road

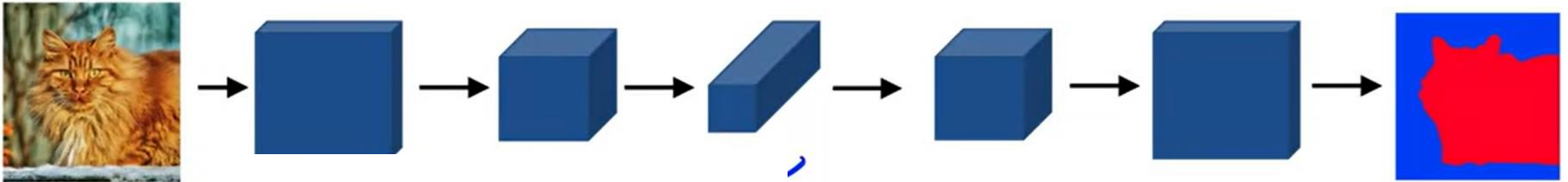


Segmentation Map

Deep Learning for Semantic Segmentation

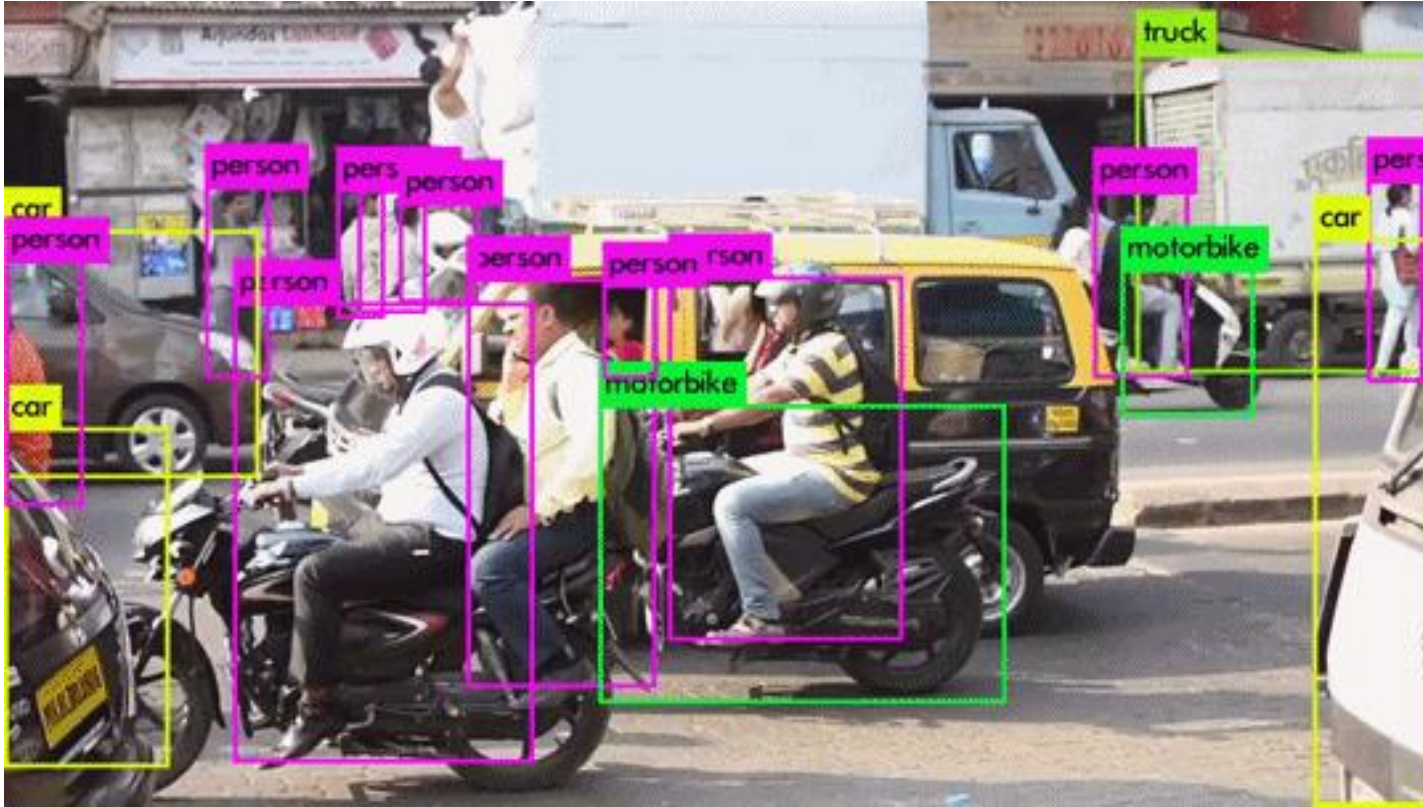


Deep Learning for Semantic Segmentation



We use transpose convolution to increase the dimension

YOLO in Action!

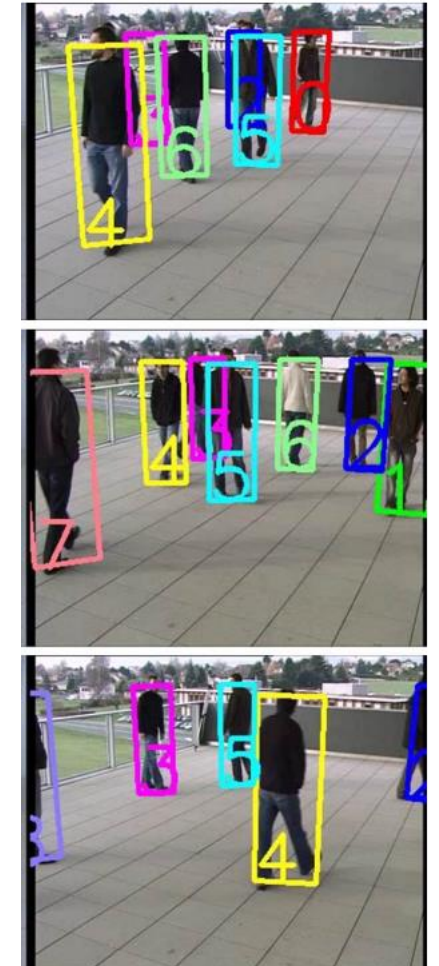
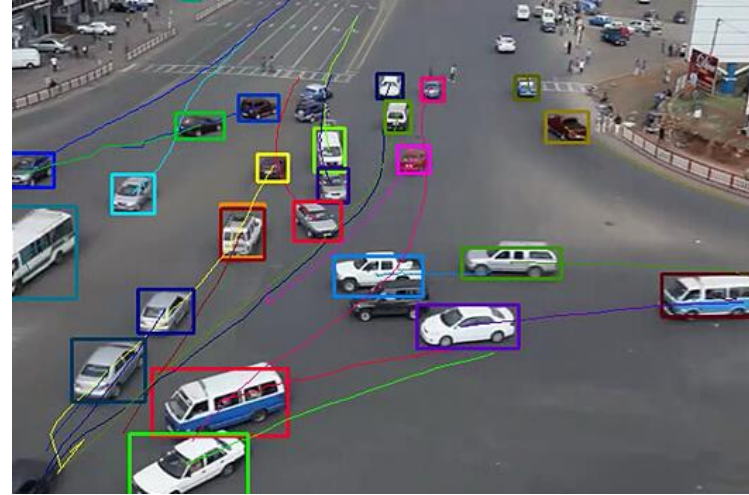


- Darknet: YOLOv ROS package



Multi Object Tracking

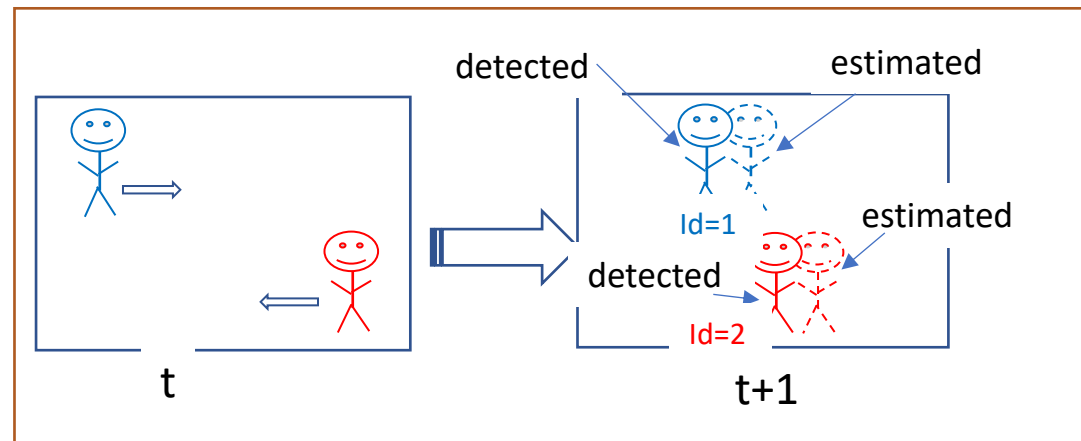
- **Goal:** track the detected objects over the time
 - assign a unique ID to each object
- Multi-object tracking is an essential task in autonomous driving applications
- Tracking:
 - Object detection
 - Data association
- **Challenge:**
 - Assign a unique ID to each tracked object.
 - MOT is difficult in cluttered environments with occluded objects



time

Multi Object Tracking

- **Traditional MOT approach:**
 - First detect the object in the current frame
 - Estimate the object's position in the next frame
 - Associate the closest detected object in the next frame to the estimated object.
- **Learning-based Tracking approaches**
 - Learn ID assignment and object detection in one-shot (e.g. FairMOT, Zhang et al. 2020)

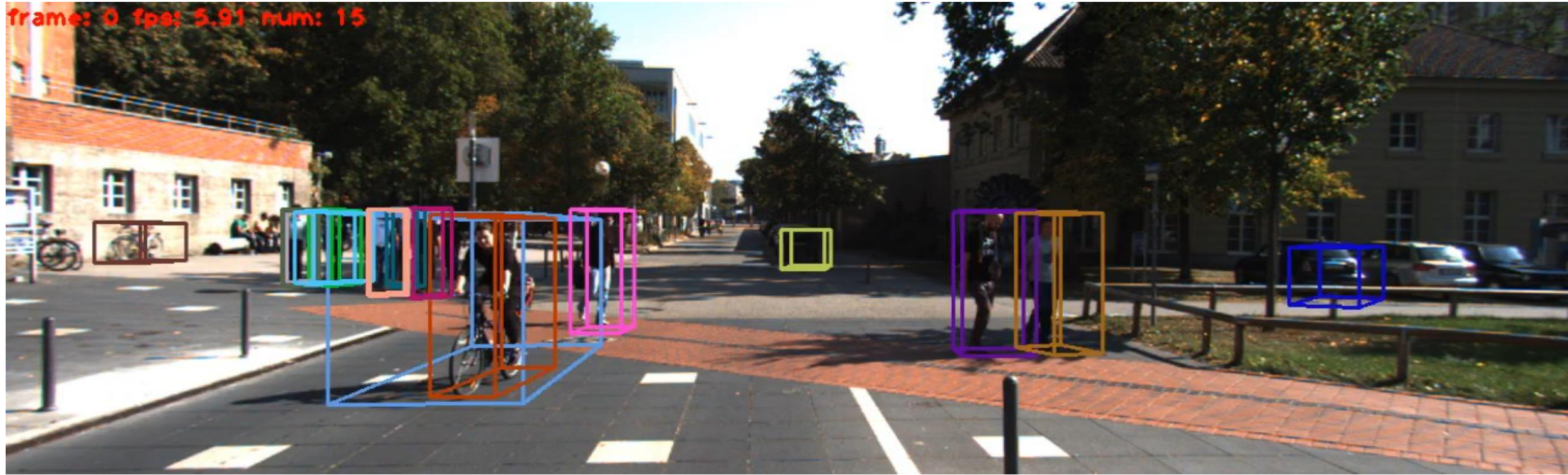


Monocular 3D multi-object Tracking

- Idea: using only image to predict object location in 3D



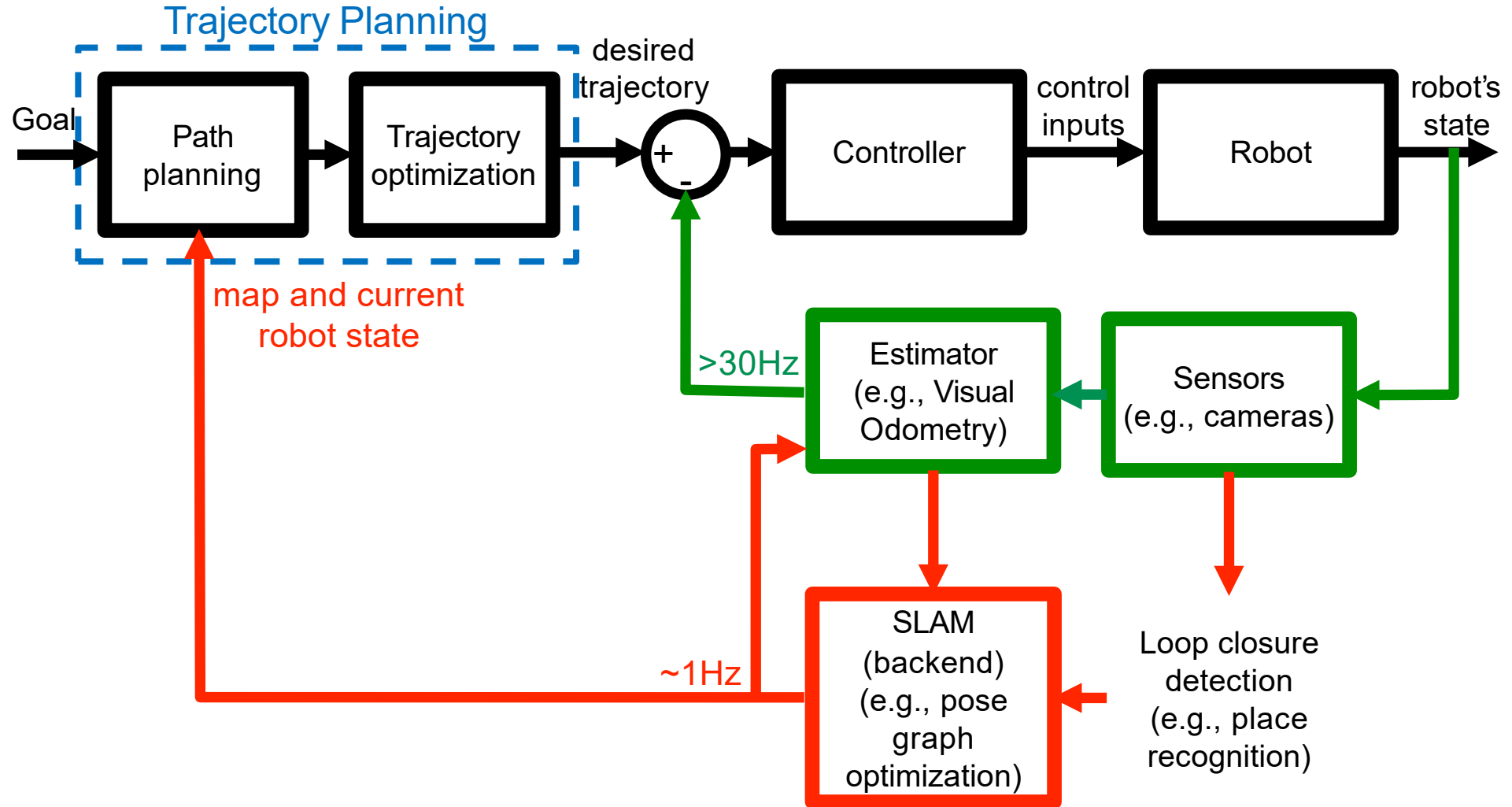
Monocular 3D MOT



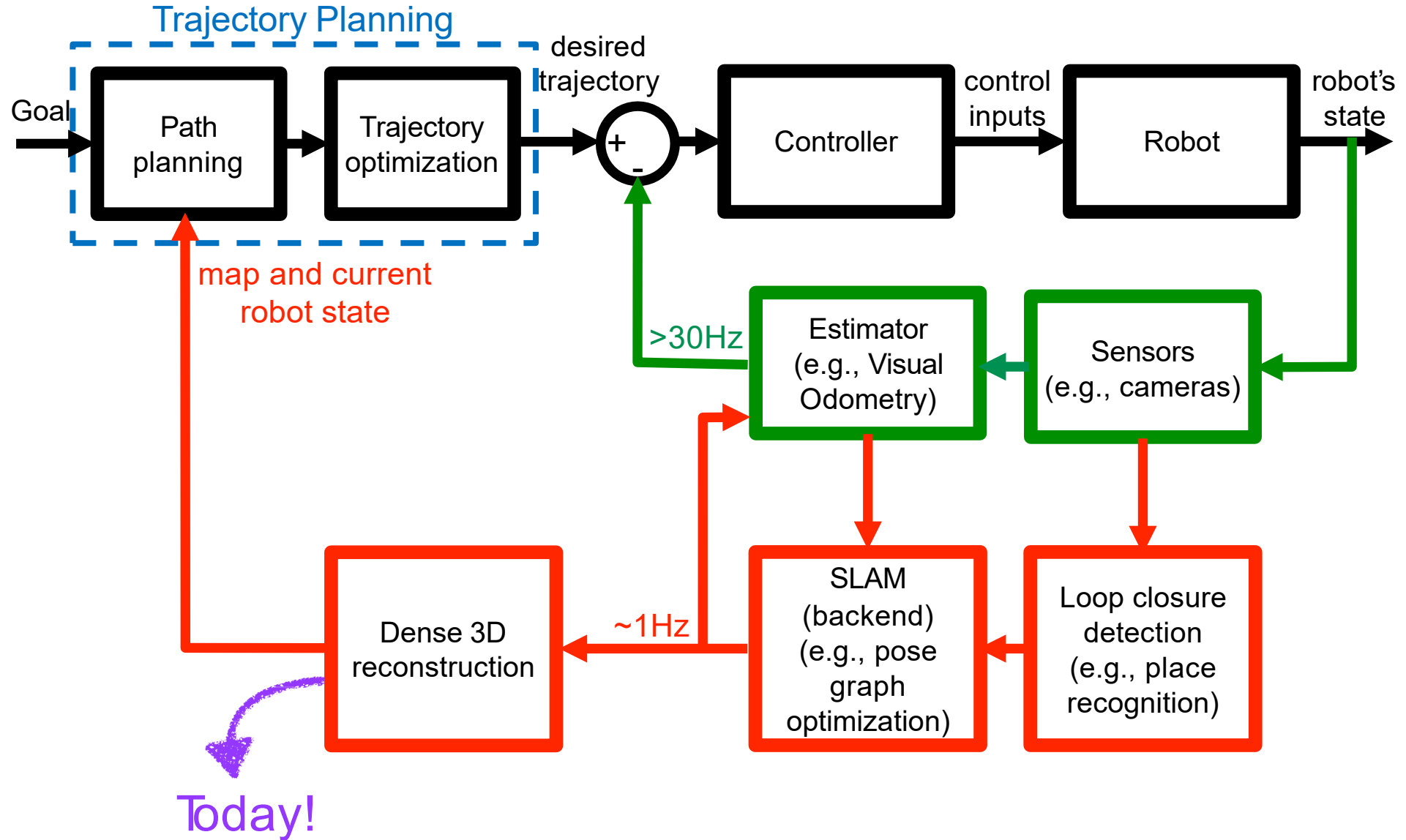
Monocular 3D MOT



Big Picture



Big Picture



Today

- Dense Reconstruction
 - 3D representations
 - (Some) Multi-view Stereo
 - Depth fusion
- Final thoughts

Multi-View Stereo: A 2015 Tutorial

Yasutaka Furukawa

Washington University in St. Louis
furukawa@wustl.edu

Carlos Hernández

Google Inc.
carloshernandez@google.com

ElasticFusion: Dense SLAM Without A Pose Graph

Thomas Whelan*, Stefan Leutenegger*, Renato F. Salas-Moreno[†], Ben Glocker[†] and Andrew J. Davison*

*Dyson Robotics Laboratory at Imperial College, Department of Computing, Imperial College London, UK

[†]Department of Computing, Imperial College London, UK

{t.whelan, s.leutenegger, r.salas-moreno10, b.glocker, a.davison}@imperial.ac.uk

2016

Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning

Helen Oleynikova, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto
Autonomous Systems Lab, ETH Zürich

Abstract—Micro Aerial Vehicles (MAVs) that operate in unstructured, unexplored environments require fast and flexible local planning, which can replan when new parts of the map are explored. Trajectory optimization methods fulfill these needs, but require obstacle distance information, which can be given by Euclidean Signed Distance Fields (ESDFs).

We propose a method to incrementally build ESDFs from Truncated Signed Distance Fields (TSDFs), a common implicit surface representation used in computer graphics and vision. TSDFs are fast to build and smooth out sensor noise over many observations, and are designed to produce surface meshes. Meshes allow human operators to get a better assessment of the robot's environment, and set high-level mission goals.



KinectFusion: Real-Time Dense Surface Mapping and Tracking*

Richard A. Newcombe
Imperial College London

Shahram Izadi
Microsoft Research

Otmar Hilliges
Microsoft Research

David Molyneux
Microsoft Research
Lancaster University

David Kim
Microsoft Research
Newcastle University

Andrew J. Davison
Imperial College London

Pushmeet Kohli
Microsoft Research

Jamie Shotton
Microsoft Research

Steve Hodges
Microsoft Research

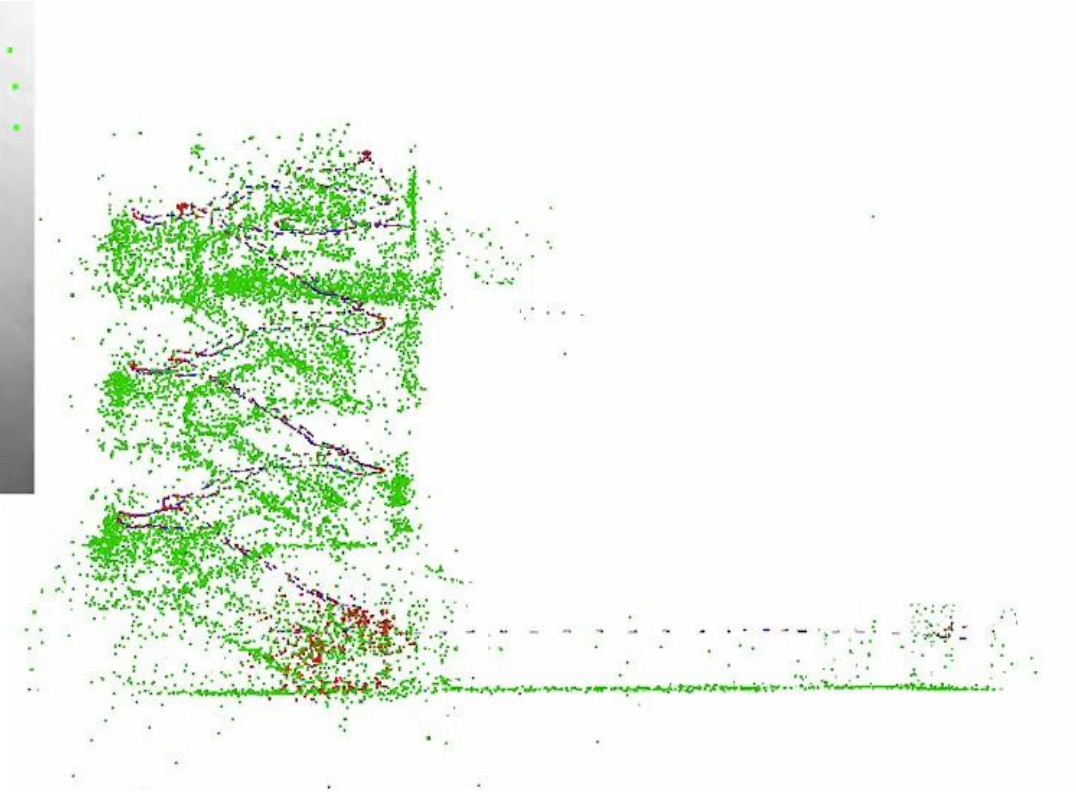
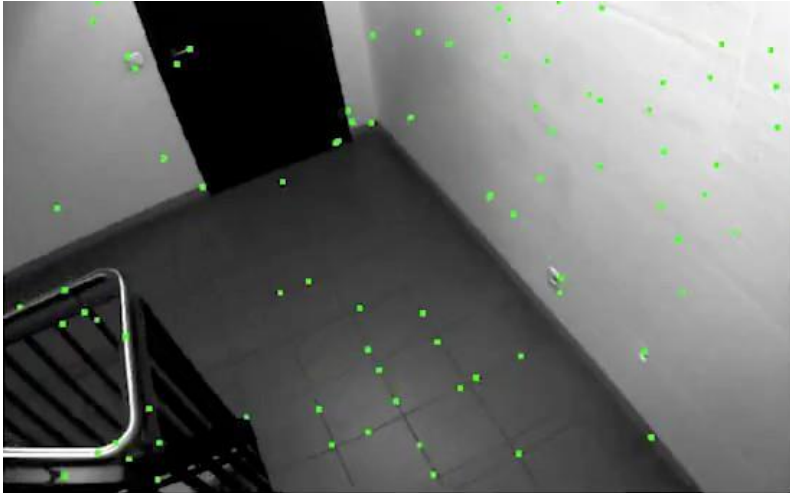
Andrew Fitzgibbon
Microsoft Research



2011

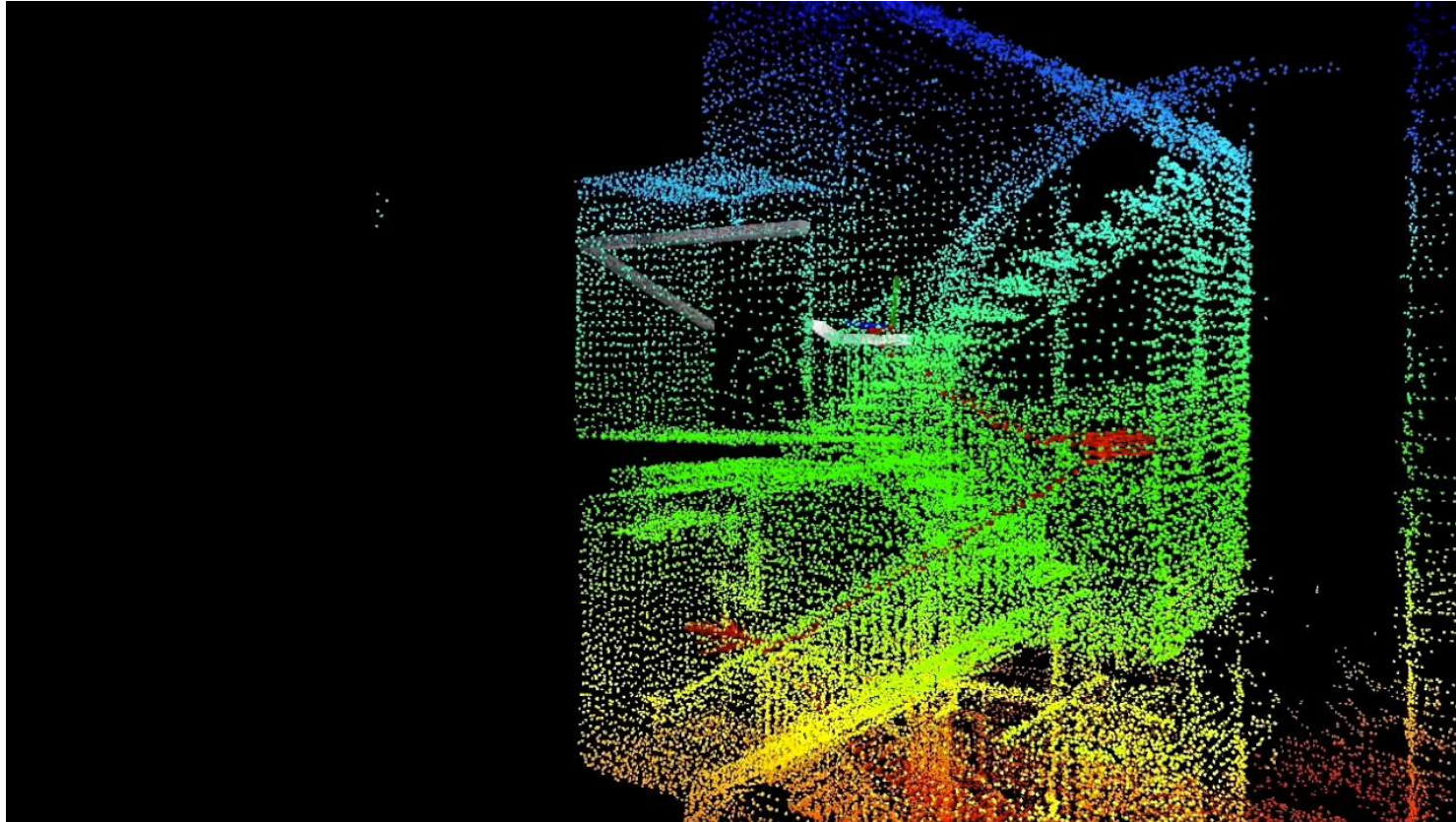
Figure 1: Example output from our system, generated in real-time with a handheld Kinect depth camera and no other sensing infrastructure. Normal maps (colour) and Phong-shaded renderings (greyscale) from our dense reconstruction system are shown. On the left for comparison is an example of the live, incomplete, and noisy data from the Kinect sensor (used as input to our system).

Point Clouds



Map representation	3D Topology?	Lightweight?	Filters Noise/Outliers?	Semantics?
Point Clouds		 No, if Dense		 No, if Sparse

Point Clouds

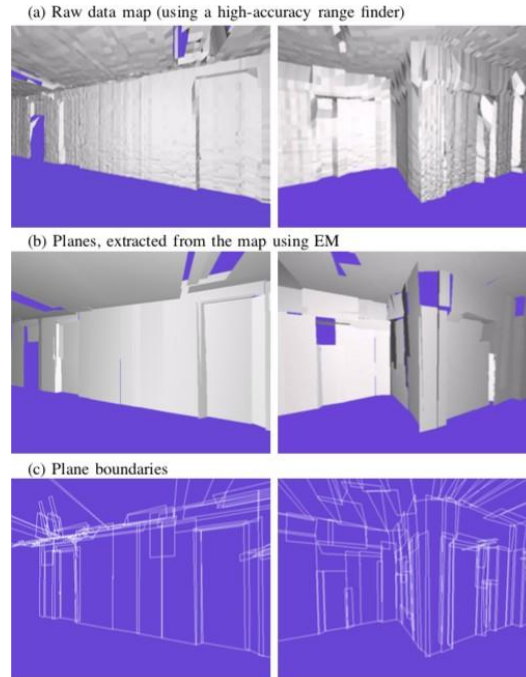


Map representation	3D Topology?	Lightweight?	Filters Noise/Outliers?	Semantics?	Generality
Point Clouds	✗	✓✗ No, if Dense	✗	✓✗ No, if Sparse	✓

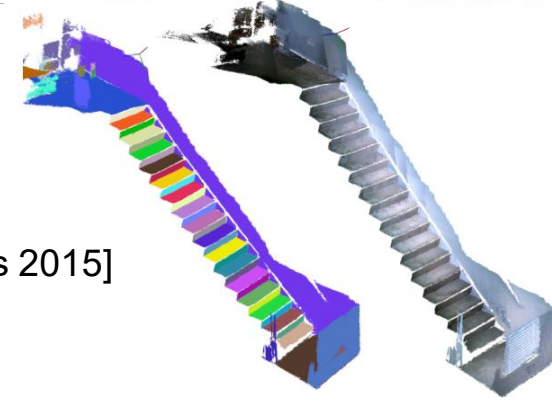
Geometric Primitives

Point, lines, planes

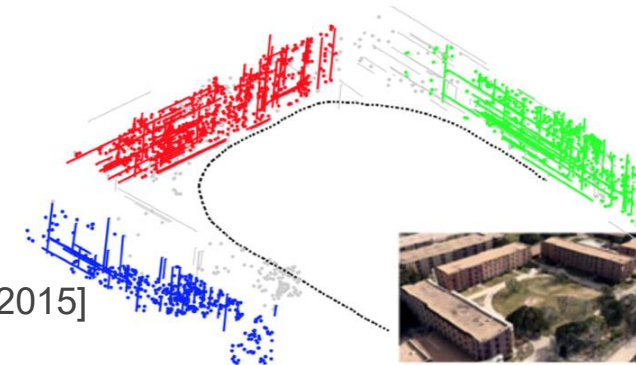
[Thrun et al.
2004]



[Kaess 2015]



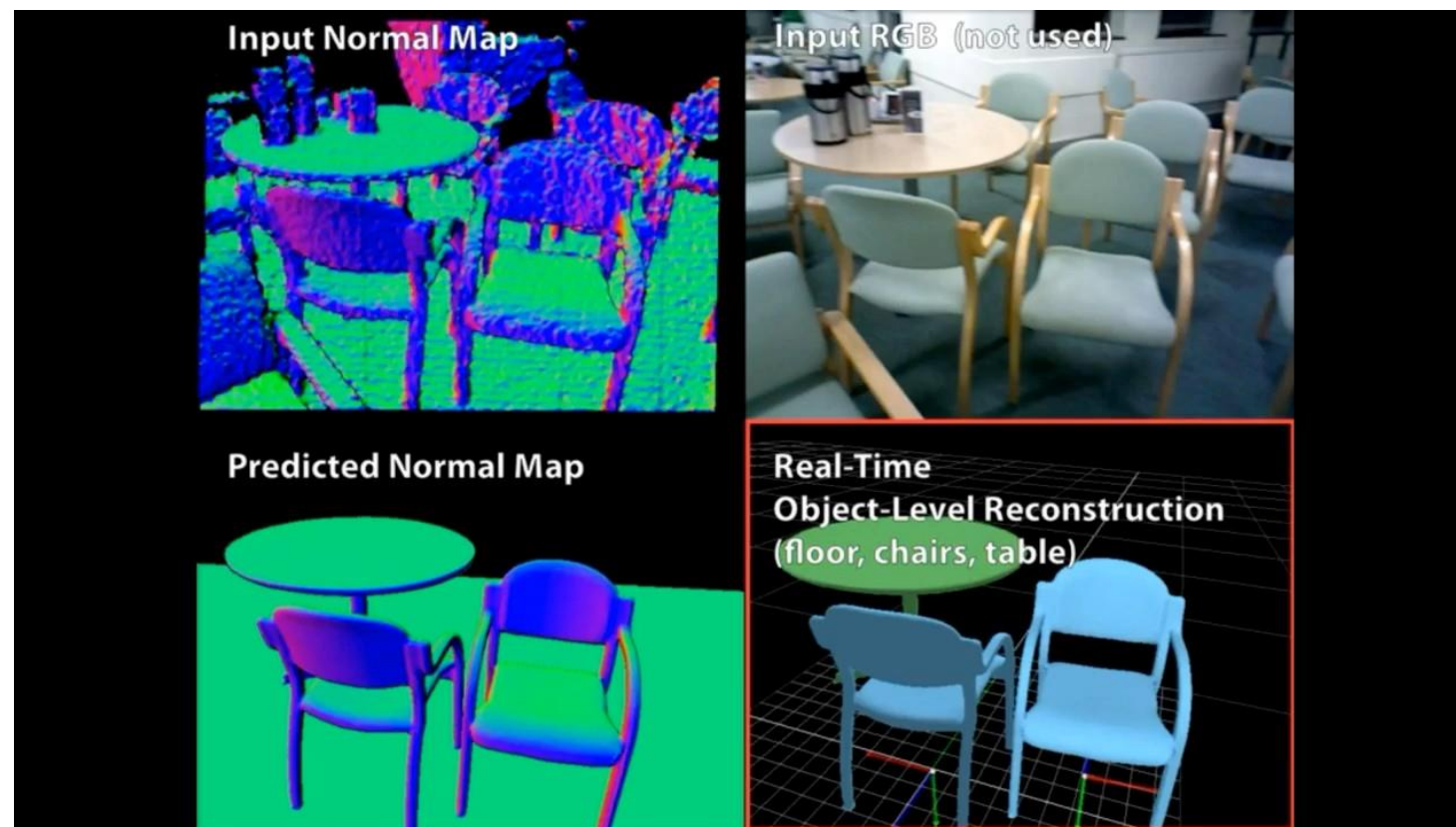
[Lu et al. 2015]



Map representation	3D Topology?	Lightweight?	Filters Noise/ Outliers?	Semantics?	Generality
Point Clouds	✗	✓/✗ No, if Dense	✗	✓/✗ No, if Sparse	✓
Geometric primitives	✗	✓	✓	✓/✗ No, if Sparse	✗

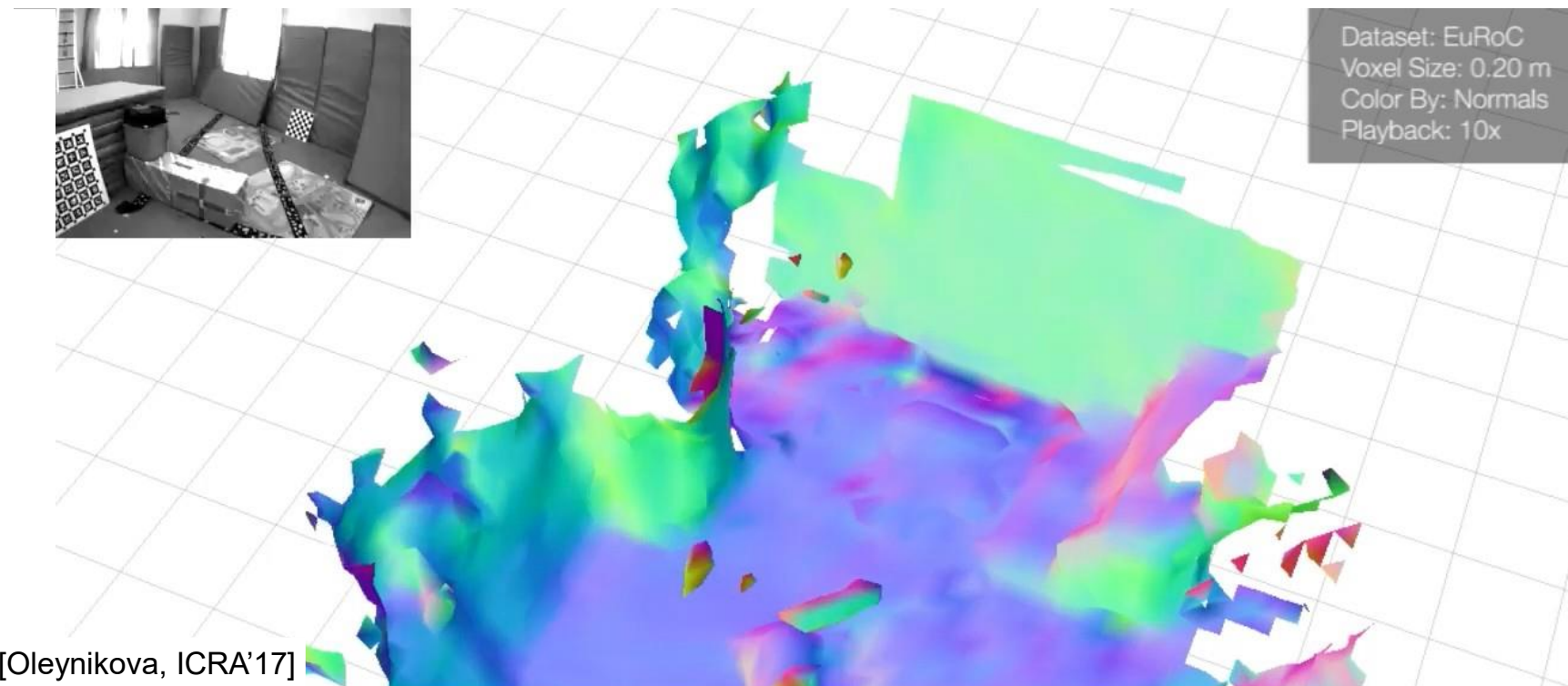
Object-based Maps

[Salas-Moreno et al, 2014]



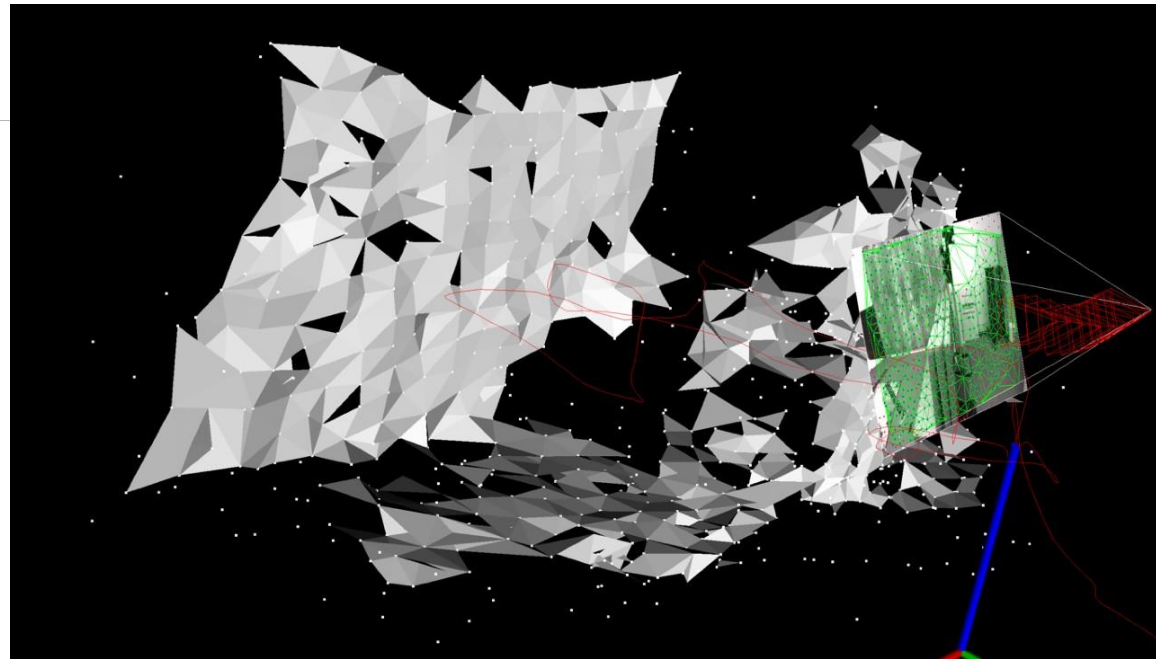
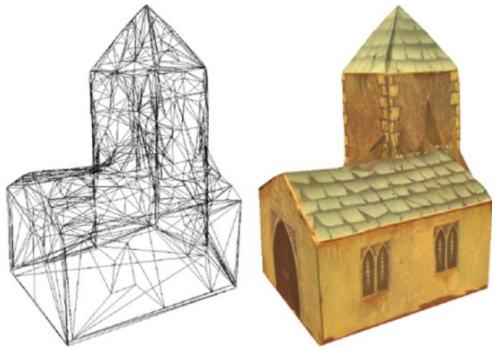
Map representation	3D Topology?	Lightweight?	Filters Noise/ Outliers?	Semantics?	Generality
Point Clouds	✗	✓/✗ No, if Dense	✗	✓/✗ No, if Sparse	✓
primitives & objects	✗	✓	✓	✓/✗ No, if Sparse	✗

Volumetric Methods: Voxels/Octrees



Map representation	3D Topology?	Lightweight?	Filters Noise/Outliers?	Semantics?	Generality
Point Clouds	✗	✓✗ No, if Dense	✗	✓✗ No, if Sparse	✓
primitives & objects	✗	✓	✓	✓✗ No, if Sparse	✗
Voxels	✓	✓✗ No, if small voxel	✓	✓✗ No, if large	✓

Meshes



Map representation	3D Topology ?	Lightweight?	Filters Noise/ Outliers?	Semantics?	Generality
Point Clouds	✗	✓/✗ No, if Dense	✗	✓/✗ No, if Sparse	✓
primitives & objects	✗	✓	✓	✓/✗ No, if Sparse	✗
Voxels	✓	✓/✗ No, if small voxel	✓	✓/✗ No, if large voxel	✓
3D Mesh	✓	✓	✗	✓	✓

Today

- Dense Reconstruction
 - 3D representations
 - (Some) Multi-view Stereo
 - Depth fusion
- Final thoughts

Multi-View Stereo: A 2015 Tutorial

Yasutaka Furukawa

Washington University in St. Louis
furukawa@wustl.edu

Carlos Hernández

Google Inc.
carloshernandez@google.com

ElasticFusion: Dense SLAM Without A Pose Graph

Thomas Whelan*, Stefan Leutenegger*, Renato F. Salas-Moreno[†], Ben Glocker[†] and Andrew J. Davison*

*Dyson Robotics Laboratory at Imperial College, Department of Computing, Imperial College London, UK

[†]Department of Computing, Imperial College London, UK

{t.whelan,s.leutenegger,r.salas-moreno10,b.glocker,a.davison}@imperial.ac.uk

2016

Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning

Helen Oleynikova, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto
Autonomous Systems Lab, ETH Zürich

Abstract—Micro Aerial Vehicles (MAVs) that operate in unstructured, unexplored environments require fast and flexible local planning, which can replan when new parts of the map are explored. Trajectory optimization methods fulfill these needs, but require obstacle distance information, which can be given by Euclidean Signed Distance Fields (ESDFs).

We propose a method to incrementally build ESDFs from Truncated Signed Distance Fields (TSDFs), a common implicit surface representation used in computer graphics and vision. TSDFs are fast to build and smooth out sensor noise over many observations, and are designed to produce surface meshes. Meshes allow human operators to get a better assessment of the robot's environment, and set high-level mission goals.



KinectFusion: Real-Time Dense Surface Mapping and Tracking*

Richard A. Newcombe
Imperial College London

Shahram Izadi
Microsoft Research

Otmar Hilliges
Microsoft Research

David Molyneux
Microsoft Research
Lancaster University

David Kim
Microsoft Research
Newcastle University

Andrew J. Davison
Imperial College London

Pushmeet Kohli
Microsoft Research

Jamie Shotton
Microsoft Research

Steve Hodges
Microsoft Research

Andrew Fitzgibbon
Microsoft Research

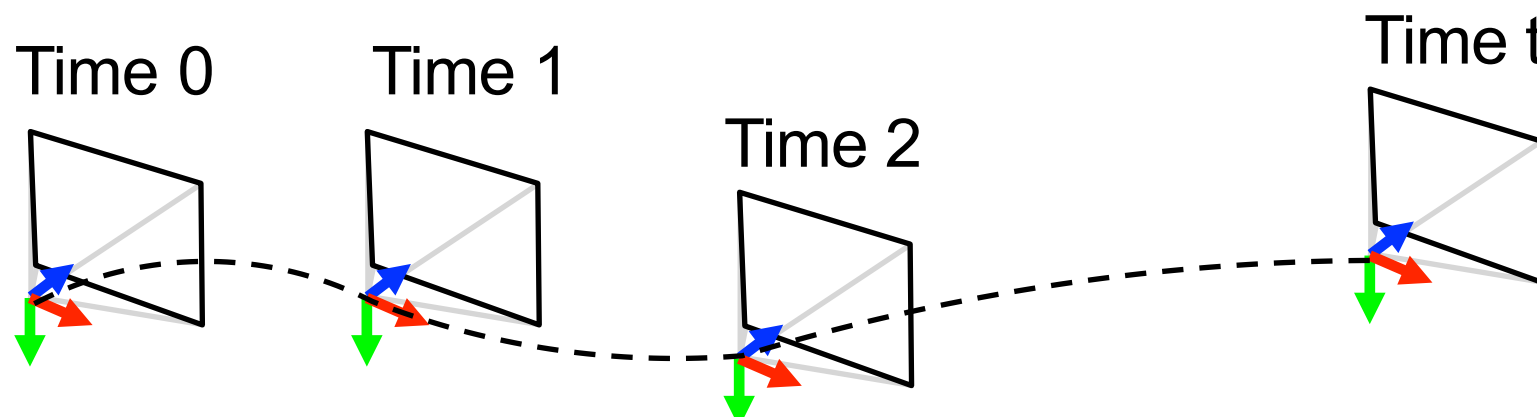


2011

Figure 1: Example output from our system, generated in real-time with a handheld Kinect depth camera and no other sensing infrastructure. Normal maps (colour) and Phong-shaded renderings (greyscale) from our dense reconstruction system are shown. On the left for comparison is an example of the live, incomplete, and noisy data from the Kinect sensor (used as input to our system).

Multi-view Stereo

From previous lectures: we know how to use SLAM to get a good estimate of the poses of the cameras



Stereo



Multi-view stereo

Multi-view Stereo

Towards Internet-scale Multi-view Stereo

CVPR 2010

Yasutaka Furukawa¹ Brian Curless²

Steven M. Seitz^{1,2} Richard Szeliski³

Google Inc.¹

University of Washington²

Microsoft Research³

Multi-view Stereo

The Visual Turing Test for Scene Reconstruction Supplementary Video

Qi Shan⁺ Riley Adams⁺ Brian Curless⁺
Yasutaka Furukawa^{*} Steve Seitz⁺⁺

⁺University of Washington ^{*}Google

3DV 2013

Multi-view Stereo

Patch-based
methods:

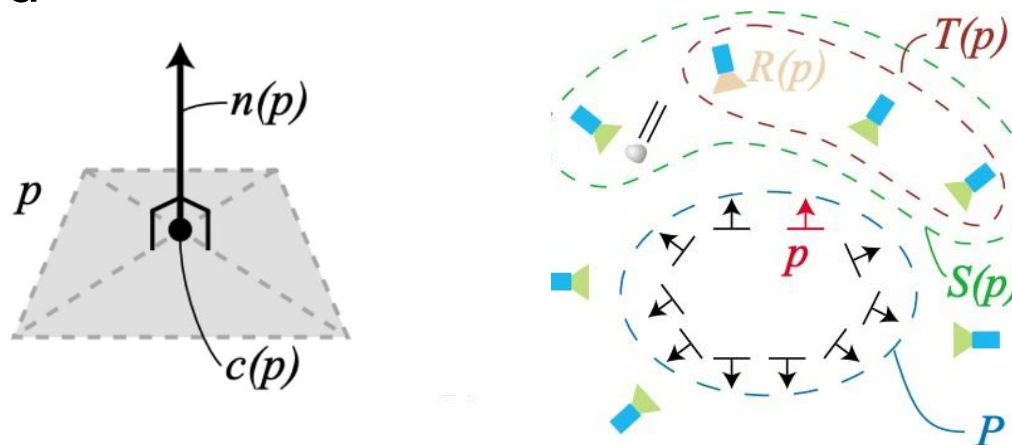


Figure 2. Definition of a patch (left) and of the images associated with it (right). See text for the details.

Estimate normal and center of patch to maximize
photometric consistency:

$$C_{ij}(p) = \rho(I_i(\Omega(\pi_i(p))), I_j(\Omega(\pi_j(p))))$$

Projection
To camera

Matching
Score

Image
Intensity

Rectangular
Patch

3D point

Example of matching score:

$$-\sum_{x,y} |W_1(x,y) - W_2(x,y)|^2$$

Multi-view Stereo

Enforcing regularity: Markov Random Fields

Find depth k_p of point “p” such that point is photo-consistent and depth changes smoothly..

$$E(\{k_p\}) = \sum_p \Phi(k_p) + \sum_{(p,q) \in \mathcal{N}} \Psi(k_p, k_q)$$

Unary potentials

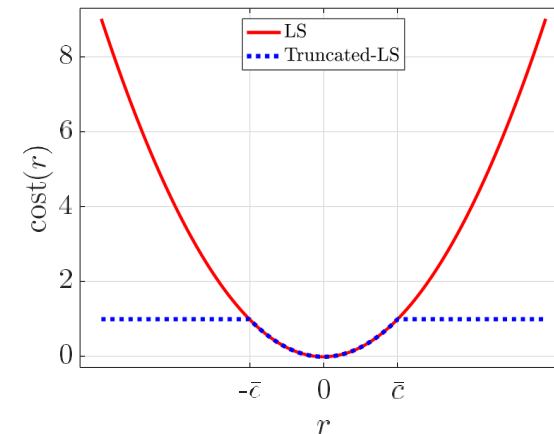
(similar to previous slides)

$$\Phi(k_p = d) = \min(\tau_u, 1 - \mathcal{C}(p, d))$$

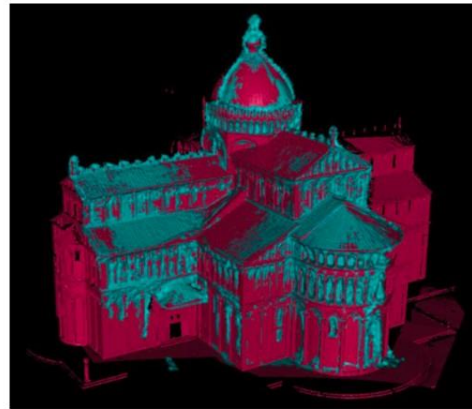
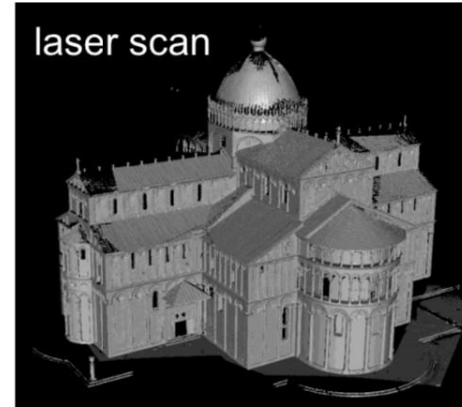
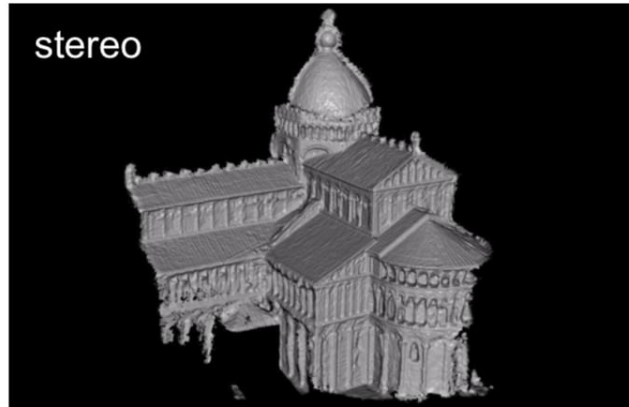
Pairwise potentials

$$\Psi(k_p = d_1, k_q = d_2) = \min(\tau_p, |d_1 - d_2|)$$

Depth is typically discretized before solving..



How Accurate is Multi-view Stereo?



Comparison: 90% of points within 0.128 m of laser scan (building height 51m)

M. Goesele, N. Snavely, B. Curless, H. Hoppe, S. Seitz, [Multi-View Stereo for Community Photo Collections](#), ICCV 2007

Space Carving Results: African Violet



Input Image (1 of 45)



Reconstruction



Reconstruction



Reconstruction

Source: S. Seitz

Space Carving Results: Hand



Input Image (1 of 100)



Views of Reconstruction

Many methods: volumetric stereo, space carving, Shape from silhouettes, carved visual hull

Today

- Dense Reconstruction
 - 3D representations
 - (Some) Multi-view Stereo
 - Depth fusion
- Final thoughts

Multi-View Stereo: A 2015 Tutorial

Yasutaka Furukawa

Washington University in St. Louis
furukawa@wustl.edu

Carlos Hernández

Google Inc.
carloshernandez@google.com

ElasticFusion: Dense SLAM Without A Pose Graph

Thomas Whelan*, Stefan Leutenegger*, Renato F. Salas-Moreno[†], Ben Glocker[†] and Andrew J. Davison*

*Dyson Robotics Laboratory at Imperial College, Department of Computing, Imperial College London, UK

[†]Department of Computing, Imperial College London, UK

{t.whelan, s.leutenegger, r.salas-moreno10, b.glocker, a.davison}@imperial.ac.uk

2016

Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning

Helen Oleynikova, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto
Autonomous Systems Lab, ETH Zürich

Abstract—Micro Aerial Vehicles (MAVs) that operate in unstructured, unexplored environments require fast and flexible local planning, which can replan when new parts of the map are explored. Trajectory optimization methods fulfill these needs, but require obstacle distance information, which can be given by Euclidean Signed Distance Fields (ESDFs).

We propose a method to incrementally build ESDFs from Truncated Signed Distance Fields (TSDFs), a common implicit surface representation used in computer graphics and vision. TSDFs are fast to build and smooth out sensor noise over many observations, and are designed to produce surface meshes. Meshes allow human operators to get a better assessment of the robot's environment, and set high-level mission goals.



KinectFusion: Real-Time Dense Surface Mapping and Tracking*

Richard A. Newcombe
Imperial College London

Shahram Izadi
Microsoft Research

Otmar Hilliges
Microsoft Research

David Molyneux
Microsoft Research
Lancaster University

David Kim
Microsoft Research
Newcastle University

Andrew J. Davison
Imperial College London

Pushmeet Kohli
Microsoft Research

Jamie Shotton
Microsoft Research

Steve Hodges
Microsoft Research

Andrew Fitzgibbon
Microsoft Research



2011

Figure 1: Example output from our system, generated in real-time with a handheld Kinect depth camera and no other sensing infrastructure. Normal maps (colour) and Phong-shaded renderings (greyscale) from our dense reconstruction system are shown. On the left for comparison is an example of the live, incomplete, and noisy data from the Kinect sensor (used as input to our system).

Surfels



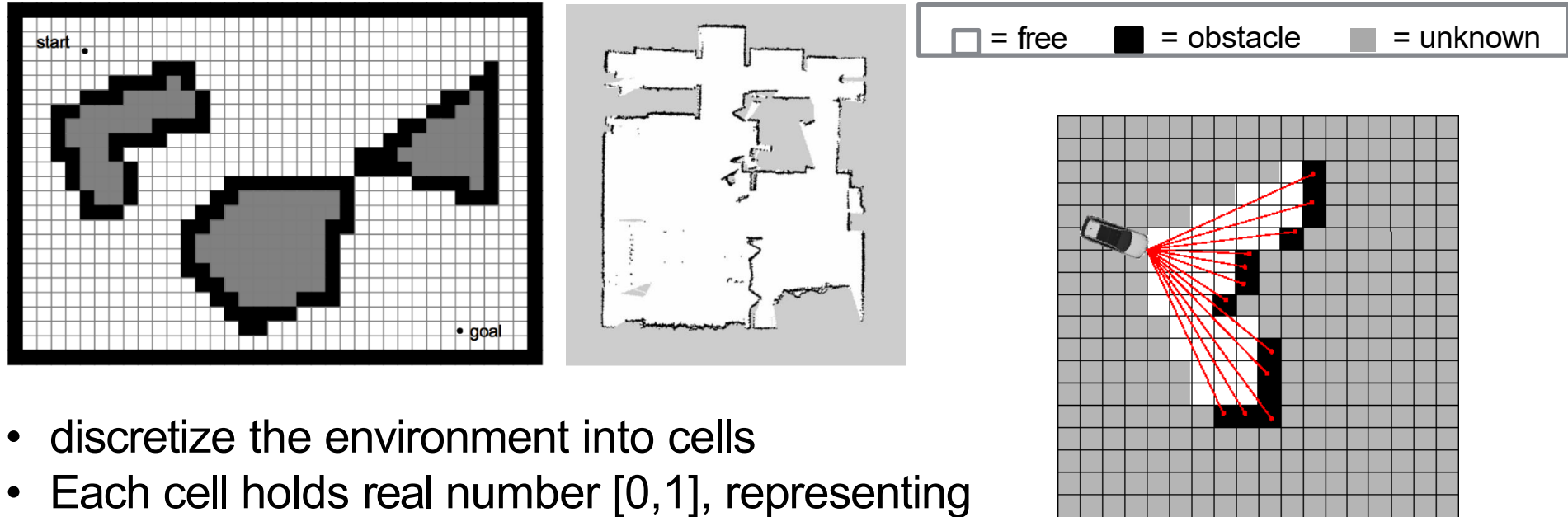
ElasticFusion: Dense SLAM Without A Pose Graph

Thomas Whelan, Stefan Leutenegger, Renato Salas-Moreno, Ben Glocker, Andrew Davison

Imperial College London

note: based on RGB-D (contrarily to multi-view stereo)

A Gentle Start: 2D Occupancy Grid Maps



- discretize the environment into cells
- Each cell holds real number $[0,1]$, representing the probability of the cell being occupied

Map posterior

$$p(m \mid z_{1:t}, x_{1:t})$$

Unknown
Map

Known sensor
Depth and robot poses

A Gentle Start: 2D Occupancy Grid Maps

$$p(m \mid z_{1:t}, x_{1:t}) \rightarrow p(\mathbf{m}_i \mid z_{1:t}, x_{1:t})$$

Binary value
(free/occupied)

Probability
of cell
being occupied

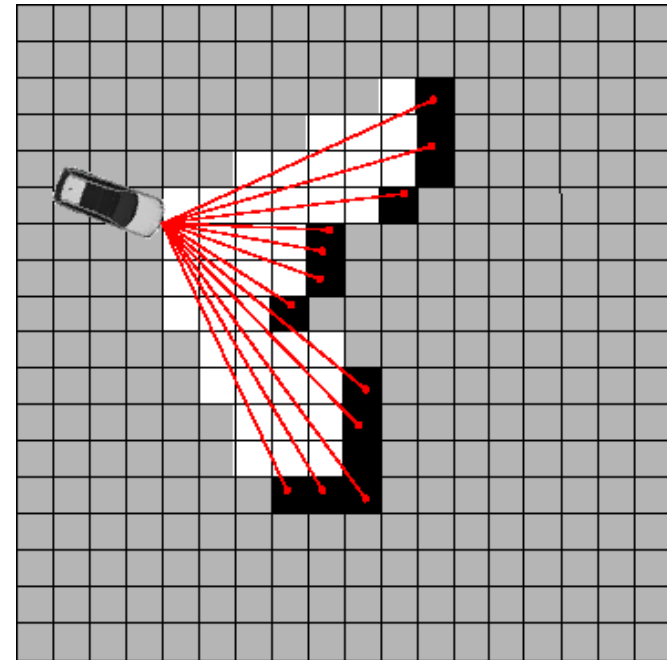
Bayes rule (omitting “x” for simplicity):

$$p(m_i \mid z_{1:t+1}) = \frac{p(z_{t+1} \mid m_i) p(m_i \mid z_{1:t})}{p(m_i)}$$

Uninformative Prior

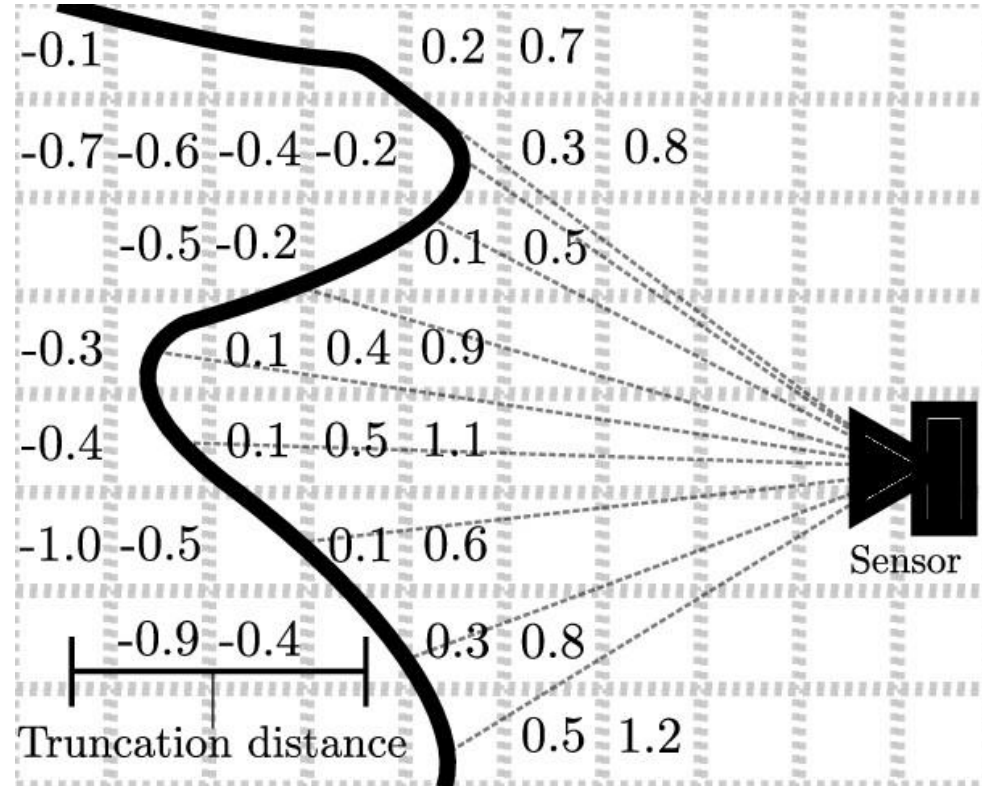
Log-odd representation is typically used to avoid numerical instabilities

$$\frac{p(\mathbf{m}_i \mid z_{1:t}, x_{1:t})}{1 - p(\mathbf{m}_i \mid z_{1:t}, x_{1:t})} \rightarrow l_{t,i} = \log \frac{p(\mathbf{m}_i \mid z_{1:t}, x_{1:t})}{1 - p(\mathbf{m}_i \mid z_{1:t}, x_{1:t})}$$



Truncated Signed Distance Function (SDF)

- Store distance to nearest obstacle (with sign)
 - Only update around obstacle itself
- (implicit surface model)



Update rule:

$$d(\mathbf{x}, \mathbf{p}, \mathbf{s}) = \|\mathbf{p} - \mathbf{x}\| \text{sign}((\mathbf{p} - \mathbf{x}) \bullet (\mathbf{p} - \mathbf{s})) \quad (1)$$

$$w_{\text{const}}(\mathbf{x}, \mathbf{p}) = 1 \quad (2)$$

$$D_{i+1}(\mathbf{x}, \mathbf{p}) = \frac{W_i(\mathbf{x})D_i(\mathbf{x}) + w(\mathbf{x}, \mathbf{p})d(\mathbf{x}, \mathbf{p})}{W_i(\mathbf{x}) + w(\mathbf{x}, \mathbf{p})} \quad (3)$$

$$W_{i+1}(\mathbf{x}, \mathbf{p}) = \min(W_i(\mathbf{x}) + w(\mathbf{x}, \mathbf{p}), W_{\text{max}}) \quad (4)$$

[Curless and Levoy, "A Volumetric Method for Building Complex Models from Range Images", 2007]

Kinect Fusion (2011)

SIGGRAPH Talks 2011

KinectFusion:

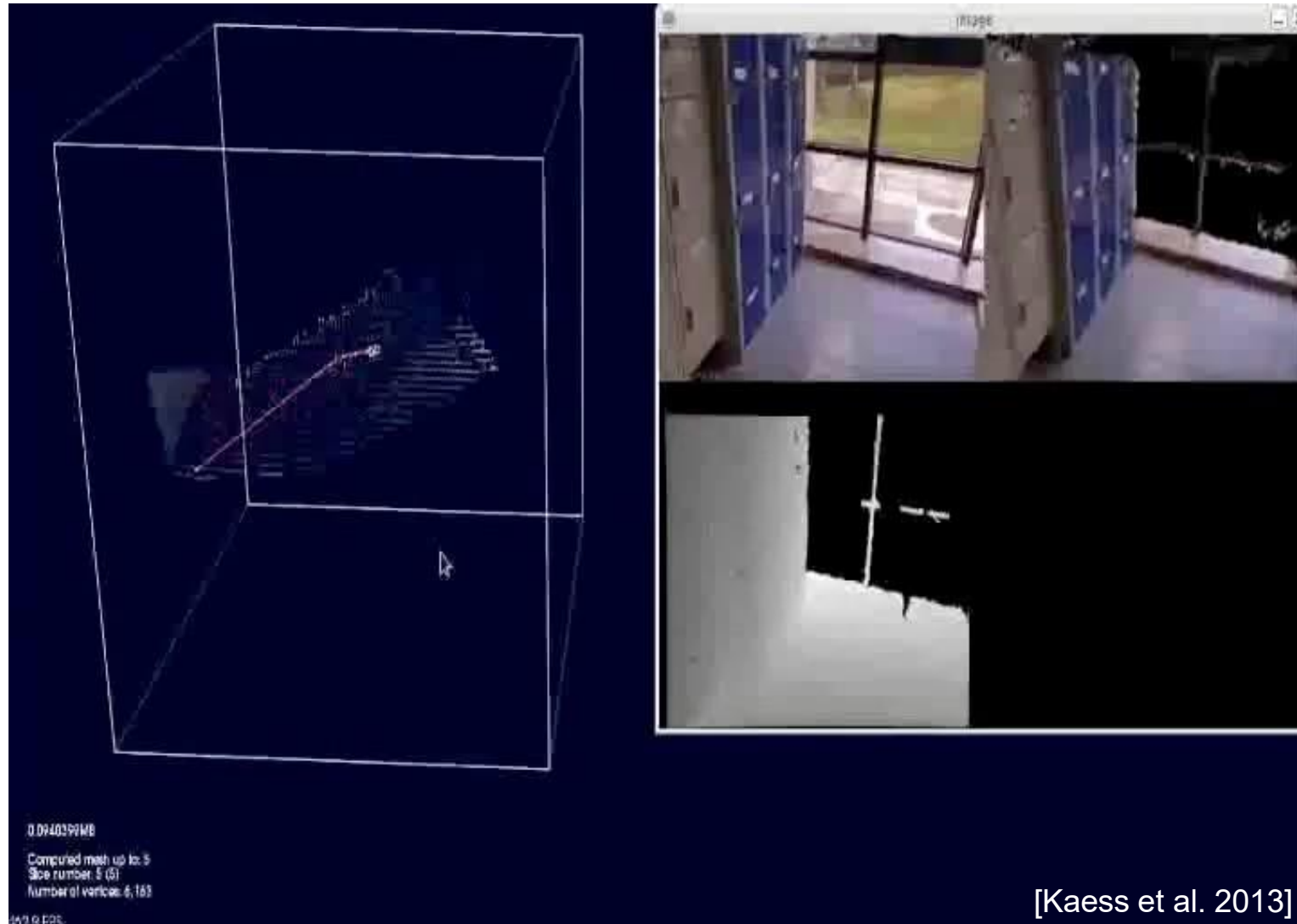
**Real-Time Dynamic 3D Surface
Reconstruction and Interaction**

**Shahram Izadi 1, Richard Newcombe 2, David Kim 1,3, Otmar Hilliges 1,
David Molyneaux 1,4, Pushmeet Kohli 1, Jamie Shotton 1,
Steve Hodges 1, Dustin Freeman 5, Andrew Davison 2, Andrew Fitzgibbon 1**

**1 Microsoft Research Cambridge 2 Imperial College London
3 Newcastle University 4 Lancaster University
5 University of Toronto**

GPU, memory ...

Kintinuous (2013)



GPU, bounded memory ...

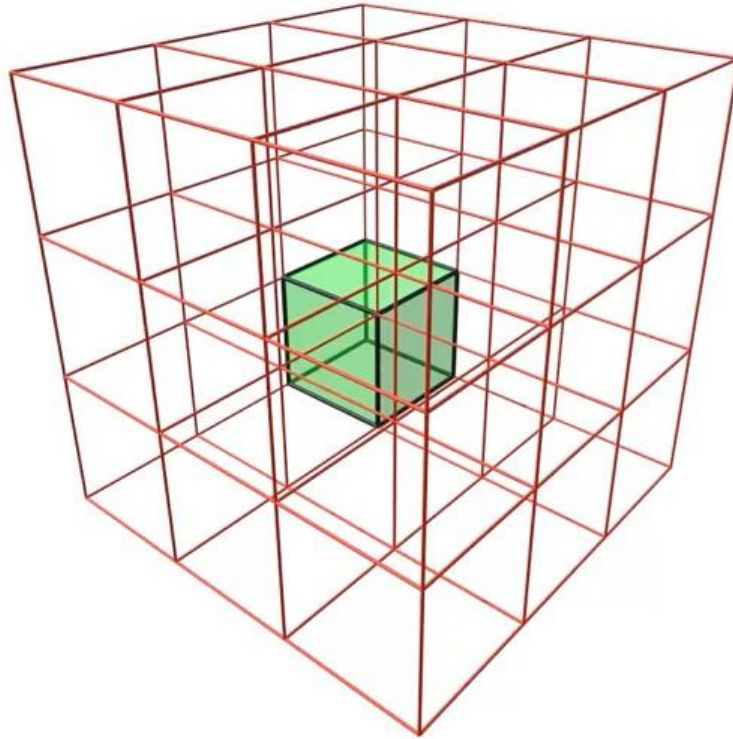
VoxBlox (2017)



CPU, memory

From Voxels to Meshes

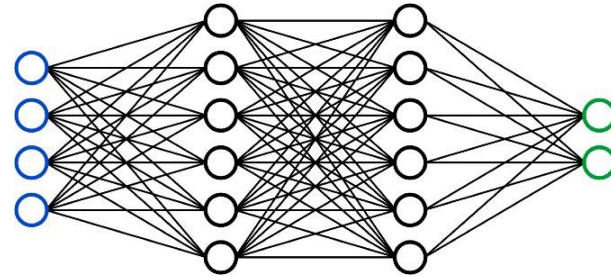
Marching cubes



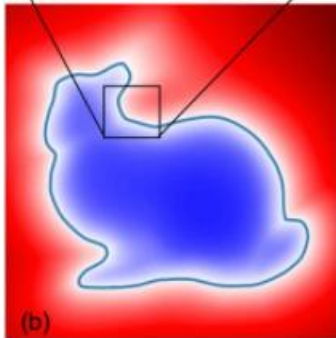
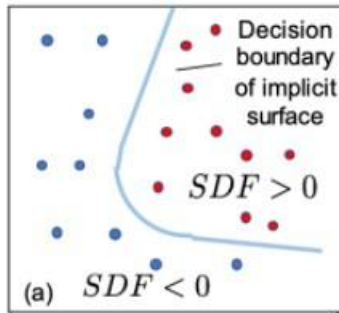
New Representations: Neural Implicit Surfaces

Use neural networks to define an **implicit** surface representation

(x, y, z)



Signed distance
To obstacles



(c)



DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation, Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove, CVPR 2019.

Neural Radiance Fields or NeRF

Use neural networks to regress not a signed distance function, but **color -> rendering**

NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis

Ben Mildenhall*
UC Berkeley

Pratul P. Srinivasan*
UC Berkeley

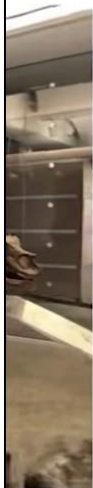
Matthew Tancik*
UC Berkeley

Jonathan T. Barron
Google Research

Ravi Ramamoorthi
UC San Diego

Ren Ng
UC Berkeley

* Denotes Equal Contribution



Neural Volume Rendering

Frank Dellaert

[Publications](#)

[Teaching](#)

[Talks](#)

[Blog Posts](#)



Frank Dellaert

Professor, Robotics &
Computer Vision

📍 Atlanta, GA

📍 Georgia Tech

✉ Email

🐦 Twitter

🌐 LinkedIn

🏠 Github

📺 YouTube

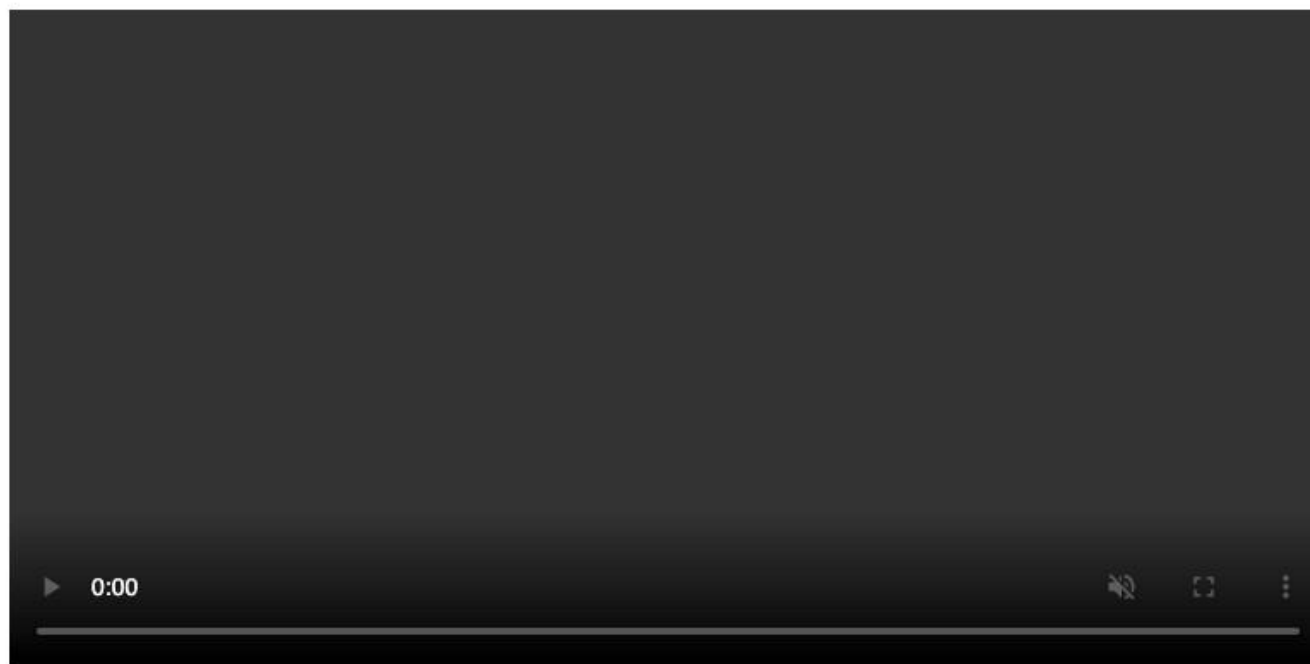
🎓 Google Scholar

🔑 ORCID

NeRF Explosion 2020

21 minute read

📅 **Published:** December 16, 2020



The result that got me hooked on wanting to know everything about NeRF :-).

Today

- Dense Reconstruction
 - 3D representations
 - (Some) Multi-view Stereo
 - Depth fusion
- Final thoughts

Multi-View Stereo: A 2015 Tutorial

Yasutaka Furukawa

Washington University in St. Louis
furukawa@wustl.edu

Carlos Hernández

Google Inc.
carloshernandez@google.com

ElasticFusion: Dense SLAM Without A Pose Graph

Thomas Whelan*, Stefan Leutenegger*, Renato F. Salas-Moreno[†], Ben Glocker[†] and Andrew J. Davison*

*Dyson Robotics Laboratory at Imperial College, Department of Computing, Imperial College London, UK

[†]Department of Computing, Imperial College London, UK

{t.whelan, s.leutenegger, r.salas-moreno10, b.glocker, a.davison}@imperial.ac.uk

2016

Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning

Helen Oleynikova, Zachary Taylor, Marius Fehr, Roland Siegwart, and Juan Nieto
Autonomous Systems Lab, ETH Zürich

Abstract—Micro Aerial Vehicles (MAVs) that operate in unstructured, unexplored environments require fast and flexible local planning, which can replan when new parts of the map are explored. Trajectory optimization methods fulfill these needs, but require obstacle distance information, which can be given by Euclidean Signed Distance Fields (ESDFs).

We propose a method to incrementally build ESDFs from Truncated Signed Distance Fields (TSDFs), a common implicit surface representation used in computer graphics and vision. TSDFs are fast to build and smooth out sensor noise over many observations, and are designed to produce surface meshes. Meshes allow human operators to get a better assessment of the robot's environment, and set high-level mission goals.



KinectFusion: Real-Time Dense Surface Mapping and Tracking*

Richard A. Newcombe
Imperial College London

Shahram Izadi
Microsoft Research

Otmar Hilliges
Microsoft Research

David Molyneaux
Microsoft Research
Lancaster University

David Kim
Microsoft Research
Newcastle University

Andrew J. Davison
Imperial College London

Pushmeet Kohli
Microsoft Research

Jamie Shotton
Microsoft Research

Steve Hodges
Microsoft Research

Andrew Fitzgibbon
Microsoft Research



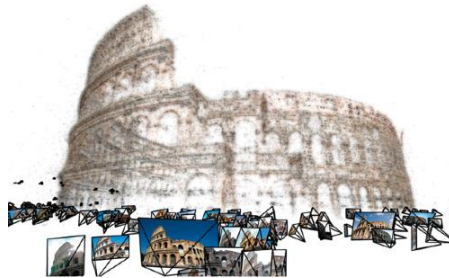
2011

Figure 1: Example output from our system, generated in real-time with a handheld Kinect depth camera and no other sensing infrastructure. Normal maps (colour) and Phong-shaded renderings (greyscale) from our dense reconstruction system are shown. On the left for comparison is an example of the live, incomplete, and noisy data from the Kinect sensor (used as input to our system).

Robot Perception or Computer Vision?

Computer vision

.. “a day on a cluster with 500 compute cores”



Robotics



50-100ms latency,
embedded,
incremental

No longer a dichotomy for many vision applications!

Robot Perception or Computer Vision?



Unordered
Vs
Sequential

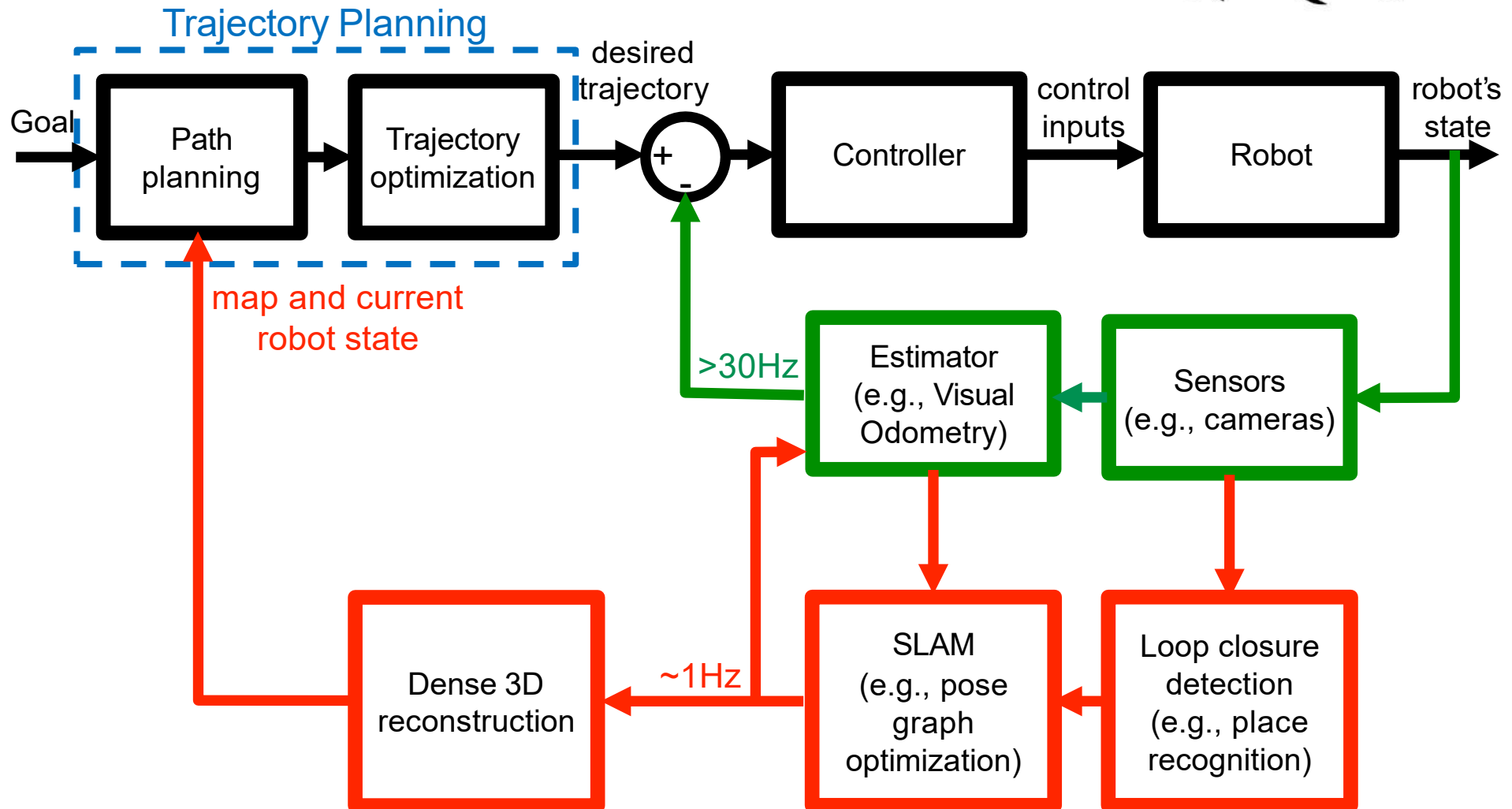
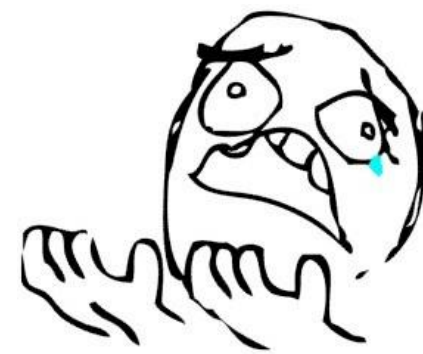


Robot Perception or Computer Vision?

Perception serves action (and vice-versa!)



But why so many steps?



Trajectory

